

Processing & Training

Alexandre Bidaux

January 13, 2026

1 Preprocessing and training

In this report, we will go over how we approached the preprocessing of our data and the training of our models.

IMPORTANT : We will not execute python code in this report. The training and preprocessing are done within the python files found in the `scripts` folder.

1.1 Preprocessing

1.1.1 What are we preprocessing for

Before starting the preprocessing of our dataset, we need to go over what we are preprocessing for. And we have 2 different cases with 3 subcases each.

The first case is when we are training for classical models (linear regression, random forest, gradient boosting, ...) and the second one is when we are preprocessing for Generalized Additive Models (GAMs).

In each one of these cases, we will also want to process our data for the 3 following cases :

1. All year
2. Only winter data
3. Only summer data

The goal is to see if models based on only data from one season will outperform a model regrouping these two seasons.

1.1.2 Preprocessing for classical models

Classical models Script : `data_preprocessing.py`

First of all, we will list you the different feature transformation we did for the classical models :

- `LOCAL_TIME` was split into 2 new features : `LOCAL_TIME_HOUR` and `LOCAL_TIME_MINUTE`
- `WEEK_DAY` was converted from a string to a number (Monday = 0, Tuesday = 1, ..., Sunday = 6)
- `ROUTE` was used to filter out the data with no real toue associated with it
- A `SEASON` variable was created with summer being the months of May through September and Winter being the rest

- `PRECIP_AMOUNT` was turned into a binary variable (either there are precipitations or there are none)
- `VISIBILITY` was turned into an ordinal categorical variable :
 - Great visibility (16 km $\leq$ visibility)
 - Correct visibility (12 km $\leq$ visibility < 16 km)
 - Poor visibility (8 km $\leq$ visibility < 12 km)
 - Very poor visibility (4 km $\leq$ visibility < 8 km)
 - No visibility (visibility < 4 km)
- `WEATHER_ENG_DESC` had its values regrouped into 3 broader categories (a single incident can belong to multiple categories) :
 - Rain (Moderate Rain, Freezing Rain, Heavy Rain)
 - Fog (Freezing Fog, Haze)
 - Snow (Moderate Snow)
- `INCIDENT` had its value regrouped into 5 broader categories :
 - Safety (Collision - TTC, Security, Emergency Services, Investigation)
 - Operational (Operations - Operator, Held By, Utilized Off Route)
 - Technical (Mechanical, Cleaning - Unsanitary)
 - External (Road Blocked - NON-TTC Collision, Diversion)
 - Other (General Delay, Vision)

Here are the different processings we did for each feature :

- Cyclical encoding : `LOCAL_TIME_HOUR`, `LOCAL_TIME_MINUTE`, `WEEK_DAY`, `LOCAL_MONTH`, `LOCAL_DAY`, `WIND_DIRECTION`
- Yeo-Johnson transformation : `WIND_SPEED`
- Standard scaling : `TEMP`, `DEW_POINT_TEMP`, `HUMIDEX`, `RELATIVE_HUMIDITY`, `STATION_PRESSURE`, `WIND_SPEED`
- Onehot encoding : `ROUTE`, `INCIDENT`, `SEASON`
- Ordinal encoding : `VISIBILITY` with the following order :
 - No visibility, Very poor visibility, Poor visibility, Correct visibility, Great visibility
- Multi label binarizing : `WEATHER_ENG_DESC_LIST`

After these treatments, we delete all NaN values.

Result file : `4_preprocessed_dataset.csv`

GAM models Script : `data_preprocessing_pygam.py`

First of all, we will list you the different feature transformation we did for the GAMs :

- `LOCAL_TIME` was split into 2 new features : `LOCAL_TIME_HOUR` and `LOCAL_TIME_MINUTE`
- `WEEK_DAY` was converted from a string to a number (Monday = 0, Tuesday = 1, ..., Sunday = 6)

- ROUTE was used to filter out the data with no real toue associated with it
- A SEASON variable was created with summer being the months of May through September and Winter being the rest
- PRECIP_AMOUNT was turned into a binary variable (either there are precipitations or there are none)
- VISIBILITY was turned into an ordinal categorical variable :
 - Great visibility (16 km $\leq$ visibility)
 - Correct visibility (12 km $\leq$ visibility < 16 km)
 - Poor visibility (8 km $\leq$ visibility < 12 km)
 - Very poor visibility (4 km $\leq$ visibility < 8 km)
 - No visibility (visibility < 4 km)
- WEATHER_ENG_DESC had its values regrouped into 3 broader categories (a single incident can belong to multiple categories) :
 - Rain (Moderate Rain, Freezing Rain, Heavy Rain)
 - Fog (Freezing Fog, Haze)
 - Snow (Moderate Snow)
- INCIDENT had its value regrouped into 5 broader categories :
 - Safety (Collision - TTC, Security, Emergency Services, Investigation)
 - Operational (Operations - Operator, Held By, Utilized Off Route)
 - Technical (Mechanical, Cleaning - Unsanitary)
 - External (Road Blocked - NON-TTC Collision, Diversion)
 - Other (General Delay, Vision)

Before preprocessing the data, we clipped the data to its 1st and 99th percentile on numerical columns to limit outliers:

```
for col in numerical_columns:
    lower_bound = df[col].quantile(0.01)
    upper_bound = df[col].quantile(0.99)
    df[col] = df[col].clip(lower=lower_bound, upper=upper_bound)
```

Here are the different processings we did for each feature :

- Cyclical encoding : LOCAL_TIME_HOUR, LOCAL_TIME_HOUR, WEEK_DAY, LOCAL_MONTH, LOCAL_DAY, WIND_DIRECTION
- Ordinal encoding : ROUTE, INCIDENT, SEASON, VISIBILITY with the following order :
 - Unkown order for ROUTE, INCIDENT, SEASON
 - No visibility, Very poor visibility, Poor visibility, Correct visibility, Great visibility
- Multi label binarizing : WEATHER_ENG_DESC_LIST

After these treatments, we delete all NaN values.

The main difference with classical models is that we do not use one hot encoding or scale our variables since GAMs do not require such treatments.

Result file : 4_preprocessed_dataset_pygam.csv

Season splitting Scripts :

- data_preprocessing_only_summer.py
- data_preprocessing_only_winter.py
- data_preprocessing_pygam_summer.py
- data_preprocessing_pygam_winter.py

To split the season, we just looked up the season variable and deleted the rows not meeting the season criteria we are looking for. We also removed the season variable from the data after this treatment.

The operation is the same in both classical and GAM preprocessing

Result files :

- 4_preprocessed_dataset_summer.py
- 4_preprocessed_dataset_winter.py
- 4_preprocessed_dataset_pygam_summer.py
- 4_preprocessed_dataset_pygam_winter.py

1.2 Model training and evaluation

1.2.1 How did we train

To train our data, we decided to go for a train, test, validation split. The proportion of the data put in each category will be specified for each different test scenario we did. We use the train data to train models, the test data to hyperoptimize the model and the validation data to finally test our hyperoptimized model.

We also used optuna to hyperoptimize our models. We hyperoptimized on 100 trials per model (except if we explicitly say that we didn't). This hyperoptimization is trying to minimize the root mean squared error (rmse). We chose this metric because, through testing, we established that it was the metric that gave us the best results while hyperoptimizing the models.

Scenario 1 - Classical models, All seasons

- **Entry data :** 4_preprocessed_dataset.csv
- **Script :** train_models.py
- **Train/Test/Validation split :** Train (60%)/Validation (20%)/Test (20%)

Decision Tree Parameters :

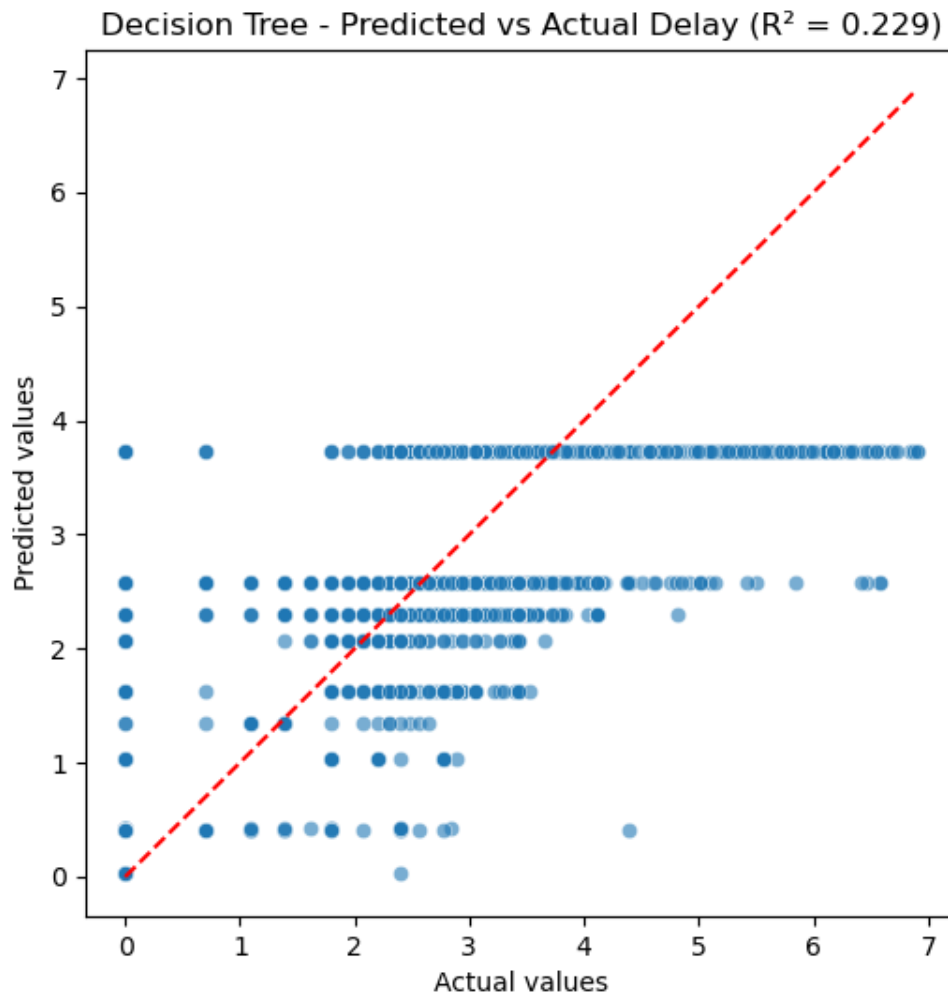
```
params = {  
    "criterion": trial.suggest_categorical("criterion", ["squared_error", "friedman_mse", "abs",  
    "splitter": trial.suggest_categorical("splitter", ["best", "random"]),  
    "max_depth": trial.suggest_int("max_depth", 1, 50),  
    "min_samples_split": trial.suggest_int("min_samples_split", 2, 20),  
    "min_samples_leaf": trial.suggest_int("min_samples_leaf", 1, 20),  
    "min_weight_fraction_leaf": trial.suggest_float("min_weight_fraction_leaf", 0.0, 0.5),  
    "max_features": trial.suggest_categorical("max_features", [None, "sqrt", "log2"]),  
    "max_leaf_nodes": trial.suggest_int("max_leaf_nodes", 2, 1000),  
    "min_impurity_decrease": trial.suggest_float("min_impurity_decrease", 0.0, 1.0),
```

```
    "ccp_alpha": trial.suggest_float("ccp_alpha", 0.0, 0.1),  
}
```

Results :

```
params = {  
    "criterion": "friedman_mse",  
    "splitter": "random",  
    "max_depth": 31,  
    "min_samples_split": 6,  
    "min_samples_leaf": 11,  
    "min_weight_fraction_leaf": 0.00014557343303502968,  
    "max_features": None,  
    "max_leaf_nodes": 861,  
    "min_impurity_decrease": 0.004493554362112649  
}
```

- **R²** : 0.2289
- **MAE** : 0.5594
- **RMSE** : 0.7146



Linear Regression (classical) Since only 4 cases are possible, only 4 trials are ran.

```
params = {
    "copy_X": True,
    "fit_intercept": trial.suggest_categorical("fit_intercept", [True, False]),
    "positive": trial.suggest_categorical("positive", [True, False]),
}
```

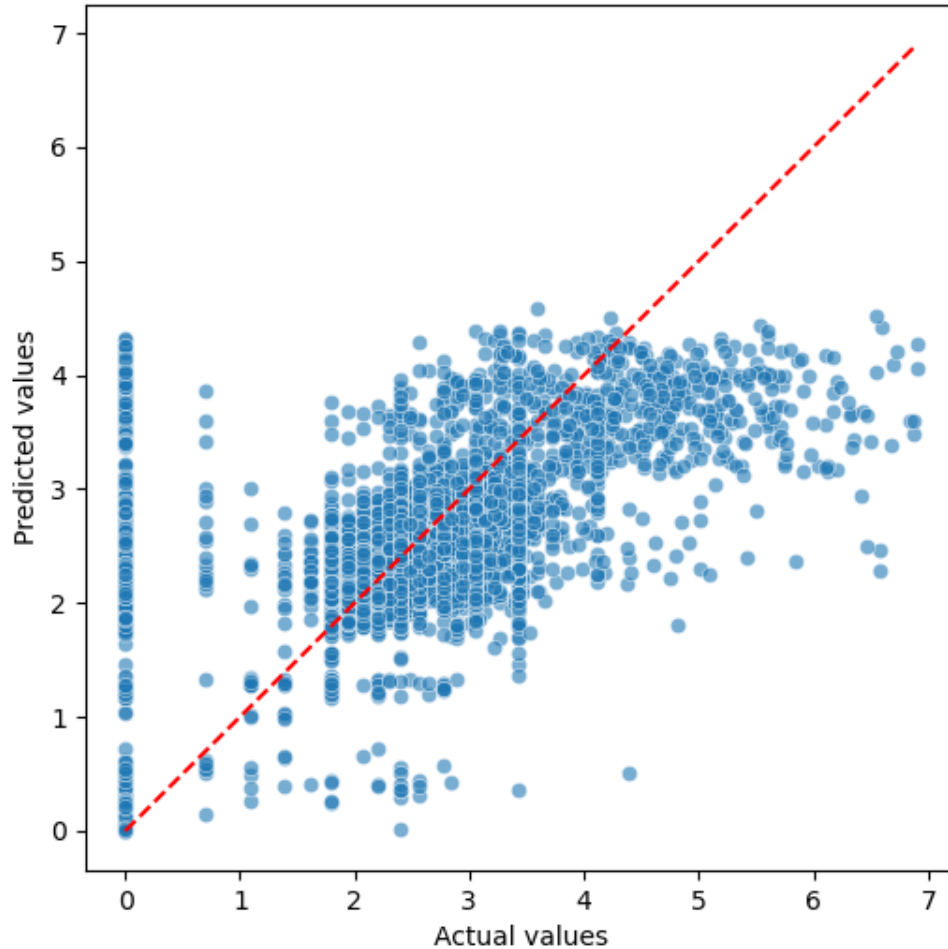
Results :

```
params = {
    "fit_intercept": False,
    "positive": True
}
```

- R^2 : 0.2921
- MAE : 0.4898

- RMSE : 0.6560

Linear Regression (classical) - Predicted vs Actual Delay ($R^2 = 0.292$)



Linear Regression (Elasticnet)

```
params = {
    "alpha": trial.suggest_float("alpha", 1e-6, 10.0, log=True),
    "l1_ratio": trial.suggest_float("l1_ratio", 0.0, 1.0),
    "fit_intercept": trial.suggest_categorical("fit_intercept", [True, False]),
    "max_iter": trial.suggest_int("max_iter", 500, 5000),
    "tol": trial.suggest_float("tol", 1e-6, 1e-2, log=True),
    "selection": trial.suggest_categorical("selection", ["cyclic", "random"])
}
```

Results :

```
params = {
    "alpha": 5.68224695093307e-05,
```

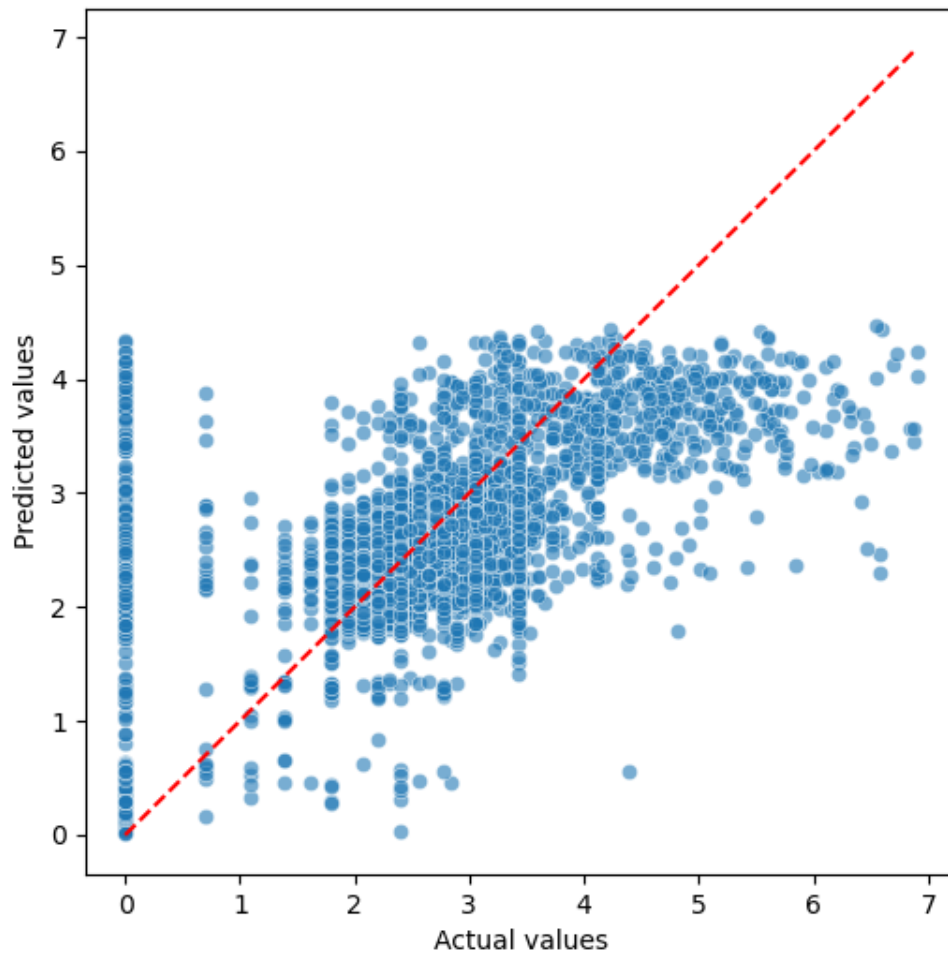
```

"l1_ratio": 0.5124624176875617,
"fit_intercept": False,
"max_iter": 1891,
"tol": 0.0027704838849413305,
"selection": "cyclic"
}

```

- R^2 : 0.2948
- MAE : 0.4902
- RMSE : 0.6535

Linear Regression (Elasticnet) - Predicted vs Actual Delay ($R^2 = 0.295$)



Random Forest We did not execute 100 trials due to extremely long execution times. We executed only 10 tests with the following parameters.

Parameters :

```
params = {
```

```

    "n_estimators": trial.suggest_int("n_estimators", 50, 600),
    "criterion": trial.suggest_categorical(
        "criterion", ["squared_error", "absolute_error", "friedman_mse", "poisson"]
    ),
    "max_depth": trial.suggest_int("max_depth", 2, 40),
    "min_samples_split": trial.suggest_int("min_samples_split", 2, 20),
    "min_samples_leaf": trial.suggest_int("min_samples_leaf", 1, 20),
    "min_weight_fraction_leaf": trial.suggest_float("min_weight_fraction_leaf", 0.0, 0.5),
    "max_features": trial.suggest_categorical(
        "max_features", ["sqrt", "log2", None]
    ),
    "max_leaf_nodes": trial.suggest_int("max_leaf_nodes", 100, 5000),
    "min_impurity_decrease": trial.suggest_float("min_impurity_decrease", 0.0, 1.0),
    "bootstrap": trial.suggest_categorical("bootstrap", [True, False]),
    "oob_score": trial.suggest_categorical("oob_score", [False, True]),
    "ccp_alpha": trial.suggest_float("ccp_alpha", 0.0, 0.1)
}

```

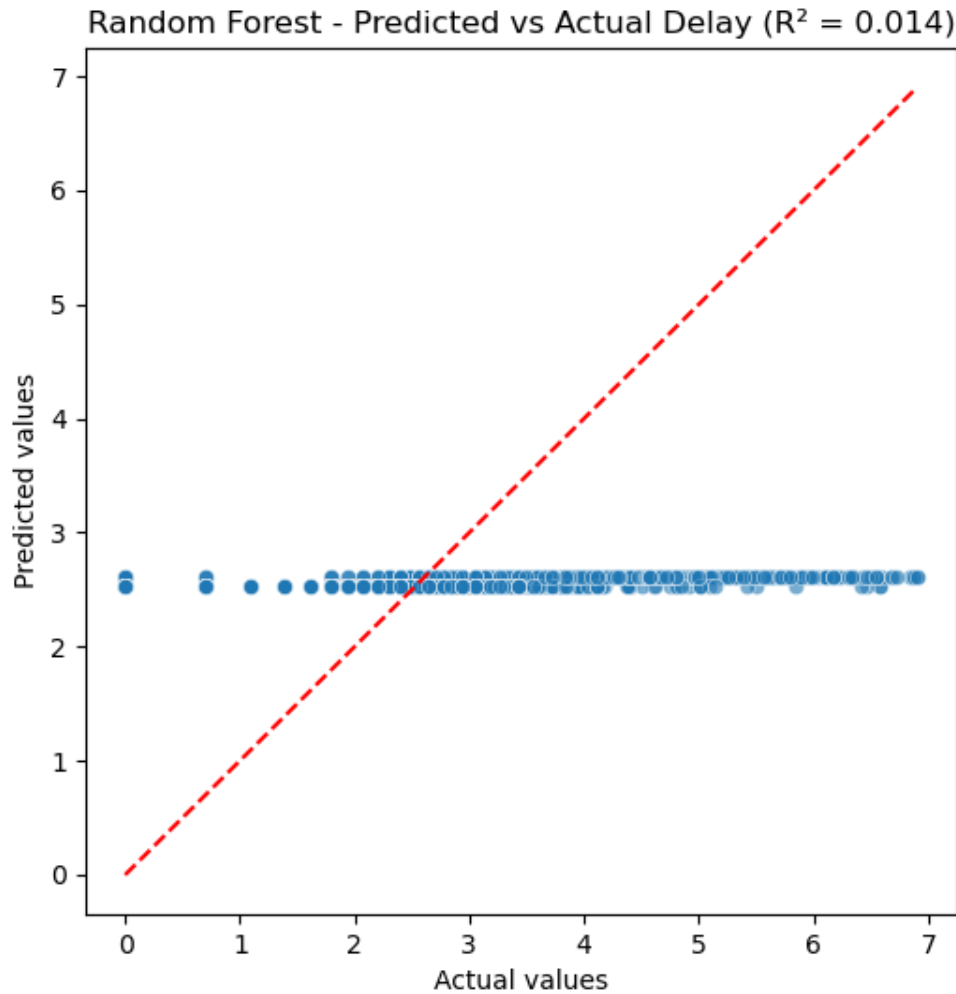
Results :

```

params = {
    "n_estimators": 149,
    "criterion": "friedman_mse",
    "max_depth": 23,
    "min_samples_split": 8,
    "min_samples_leaf": 20,
    "min_weight_fraction_leaf": 0.007770028157936426,
    "max_features": "sqrt",
    "max_leaf_nodes": 798,
    "min_impurity_decrease": 0.7201417309527923,
    "bootstrap": True,
    "oob_score": True,
    "ccp_alpha": 0.06497372551826268
}

```

- **R²** : 0.0140
- **MAE** : 0.6220
- **RMSE** : 0.9137



Gradient Boosting Parameters :

```
params = {
    "loss": trial.suggest_categorical(
        "loss", ["squared_error", "absolute_error", "huber", "quantile"]
    ),
    "learning_rate": trial.suggest_float("learning_rate", 0.001, 0.3, log=True),
    "n_estimators": trial.suggest_int("n_estimators", 50, 800),
    "subsample": trial.suggest_float("subsample", 0.5, 1.0),
    "criterion": trial.suggest_categorical(
        "criterion", ["friedman_mse", "squared_error"]
    ),
    "min_samples_split": trial.suggest_int("min_samples_split", 2, 20),
    "min_samples_leaf": trial.suggest_int("min_samples_leaf", 1, 20),
    "min_weight_fraction_leaf": trial.suggest_float("min_weight_fraction_leaf", 0.0, 0.3),
    "max_depth": trial.suggest_int("max_depth", 2, 10),
}
```

```

    "max_features": trial.suggest_categorical(
        "max_features", ["sqrt", "log2", None]
    ),
    "min_impurity_decrease": trial.suggest_float("min_impurity_decrease", 0.0, 1.0),
    "alpha": trial.suggest_float("alpha", 0.01, 0.99), # used for quantile/huber losses
}

```

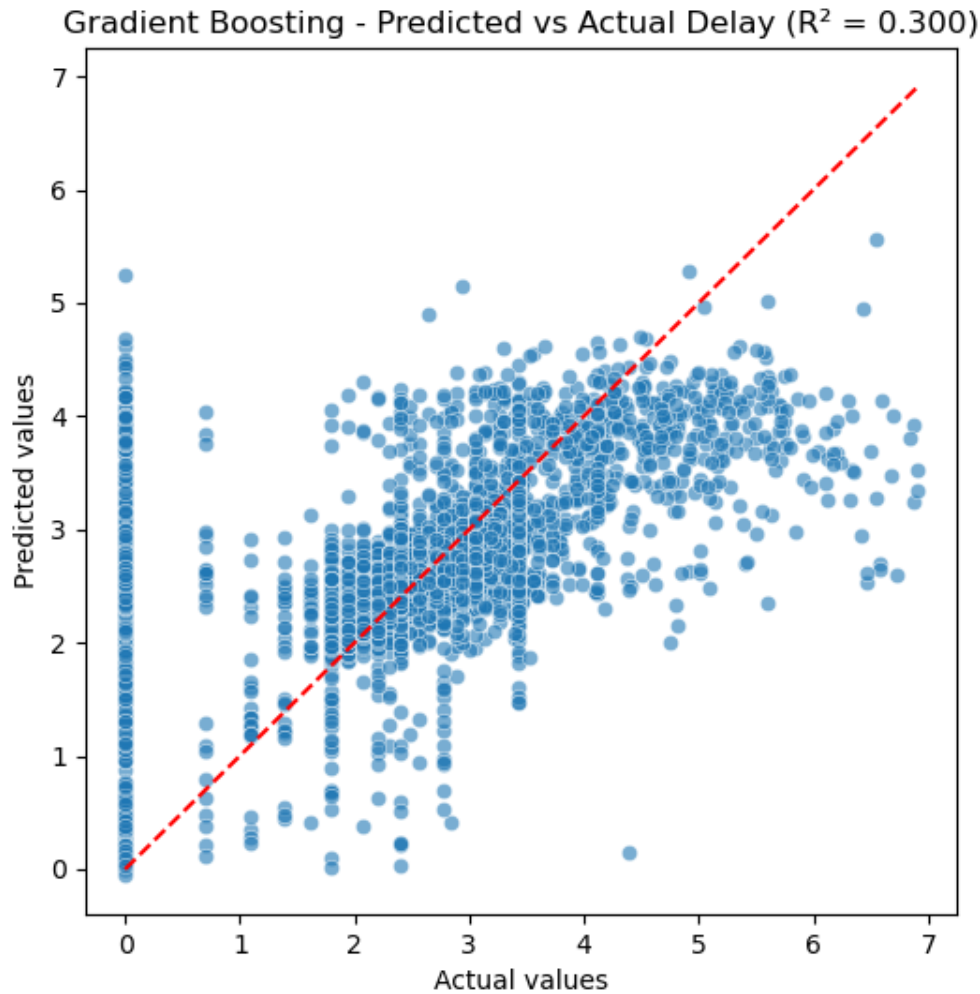
Results :

```

params = {
    "loss": "huber",
    "learning_rate": 0.03471075642674359,
    "n_estimators": 540,
    "subsample": 0.8855618217015933,
    "criterion": "friedman_mse",
    "min_samples_split": 15,
    "min_samples_leaf": 10,
    "min_weight_fraction_leaf": 0.00011386163288458159,
    "max_depth": 9,
    "max_features": None,
    "min_impurity_decrease": 0.6129026132279136,
    "alpha": 0.8665875204701363
}

```

- **R²** : 0.3002
- **MAE** : 0.4465
- **RMSE** : 0.6485



1.2.2 Extreme Gradient Boosting

Parameters :

```
params = {
    "n_estimators": trial.suggest_int("n_estimators", 200, 1500),
    "learning_rate": trial.suggest_float("learning_rate", 1e-4, 0.3, log=True),
    # Tree parameters
    "max_depth": trial.suggest_int("max_depth", 2, 12),
    "min_child_weight": trial.suggest_float("min_child_weight", 1e-3, 10.0, log=True),
    "gamma": trial.suggest_float("gamma", 0.0, 10.0),
    "max_delta_step": trial.suggest_int("max_delta_step", 0, 10),
    # Regularization
    "reg_alpha": trial.suggest_float("reg_alpha", 1e-8, 10.0, log=True),
    "reg_beta": trial.suggest_float("reg_lambda", 1e-8, 10.0, log=True),
    # Subsampling
```

```

"subsample": trial.suggest_float("subsample", 0.5, 1.0),
"colsample_bytree": trial.suggest_float("colsample_bytree", 0.5, 1.0),
"colsample_bylevel": trial.suggest_float("colsample_bylevel", 0.5, 1.0),
"colsample_bynode": trial.suggest_float("colsample_bynode", 0.5, 1.0),
# Booster & misc.
"booster": trial.suggest_categorical("booster", ["gbtree"]),
"tree_method": "auto",
"random_state": 99,
"n_jobs": -1,
}

```

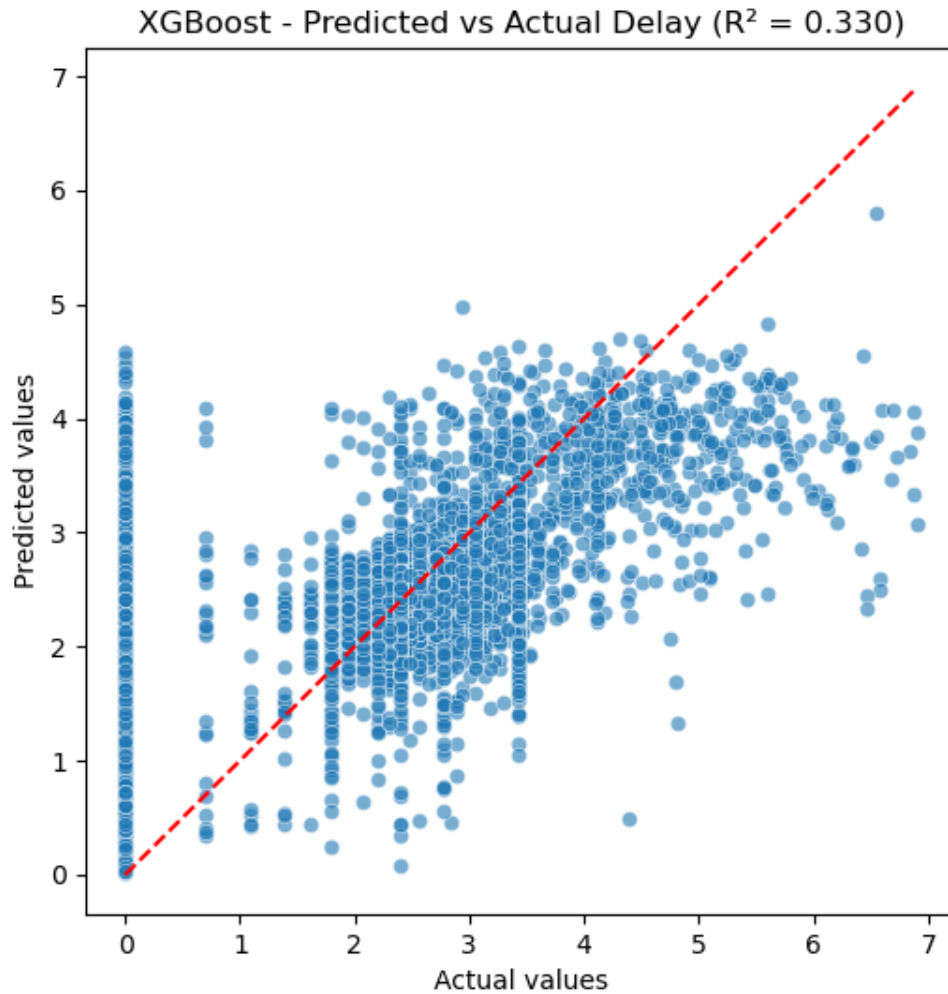
Results :

```

params = {
    "n_estimators": 1176,
    "learning_rate": 0.018310088659669047,
    "max_depth": 10,
    "min_child_weight": 0.2433298670322095,
    "gamma": 1.081134221399496,
    "max_delta_step": 3,
    "reg_alpha": 7.084876626164422e-05,
    "reg_lambda": 9.599288655044234,
    "subsample": 0.6606285902180087,
    "colsample_bytree": 0.604577745827854,
    "colsample_bylevel": 0.9379922099663431,
    "colsample_bynode": 0.540463702894869,
    "booster": "gbtree"
}

```

- R^2 : 0.3296
- MAE : 0.4746
- RMSE : 0.6212



Support Vector Regression (SVR) We did not execute 100 trials due to extremely long execution times. We executed only 5 tests with the following parameters.

Parameters :

```
kernel = trial.suggest_categorical("kernel", ["rbf", "poly", "sigmoid", "linear"])
params = {
    "kernel": kernel,
    "C": trial.suggest_float("C", 1e-3, 1e3, log=True),
    "epsilon": trial.suggest_float("epsilon", 1e-4, 1.0, log=True),
    "shrinking": trial.suggest_categorical("shrinking", [True, False]),
    "gamma": trial.suggest_categorical("gamma", ["scale", "auto"]),
}
# Kernel-specific parameters
if kernel == "poly":
    params["degree"] = trial.suggest_int("degree", 2, 5)
```

```

    params["coef0"] = trial.suggest_float("coef0", 0.0, 1.0)
elif kernel == "sigmoid":
    params["coef0"] = trial.suggest_float("coef0", 0.0, 1.0)

```

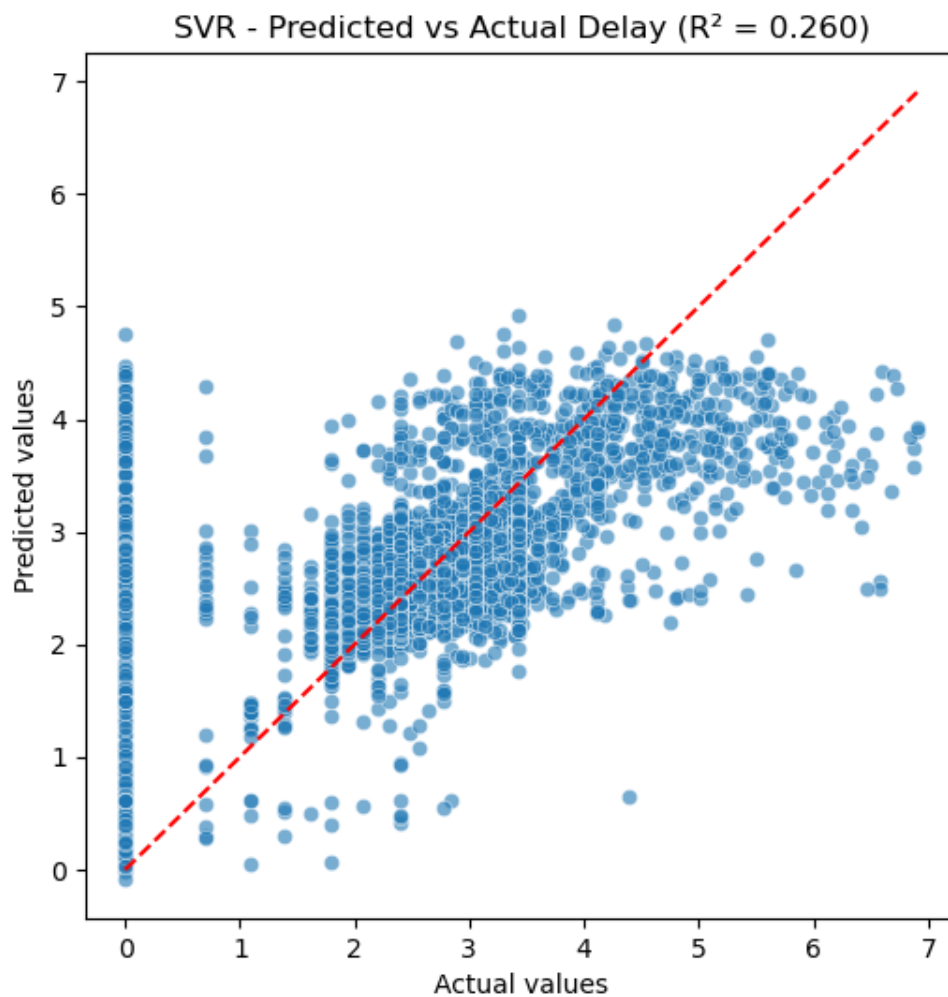
Results :

```

params = {
    "kernel": "rbf",
    "C": 0.9172634807099763,
    "epsilon": 0.022277307122931626,
    "shrinking": False,
    "gamma": "scale"
}

```

- R^2 : 0.2601
- MAE : 0.4451
- RMSE : 0.6857



Scenario 2 - Classical models, Winter

- **Entry data** : 4_preprocessed_dataset_winter.csv
- **Script** : train_models_winter.py
- **Train/Test/Validation split** : Train (60%)/Validation (20%)/Test (20%)

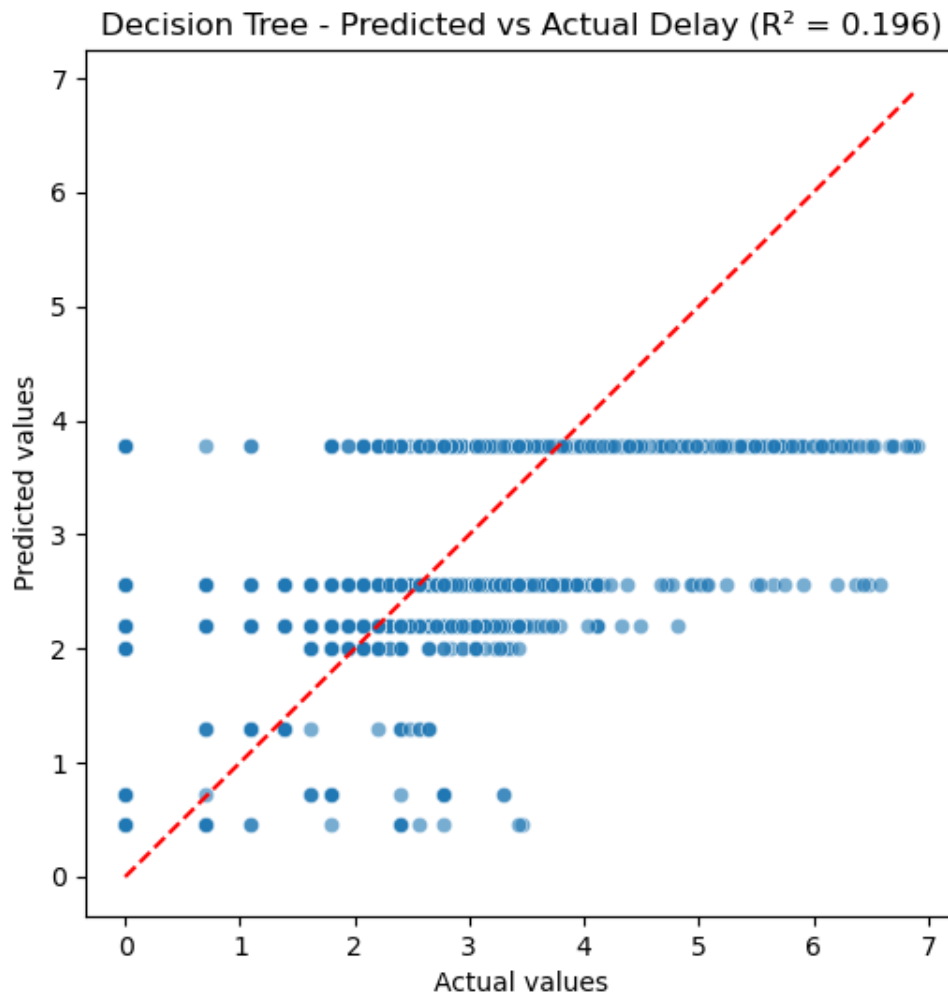
Decision Tree Parameters :

```
params = {  
    "criterion": trial.suggest_categorical("criterion", ["squared_error", "friedman_mse", "abs",  
    "splitter": trial.suggest_categorical("splitter", ["best", "random"]),  
    "max_depth": trial.suggest_int("max_depth", 1, 50),  
    "min_samples_split": trial.suggest_int("min_samples_split", 2, 20),  
    "min_samples_leaf": trial.suggest_int("min_samples_leaf", 1, 20),  
    "min_weight_fraction_leaf": trial.suggest_float("min_weight_fraction_leaf", 0.0, 0.5),  
    "max_features": trial.suggest_categorical("max_features", [None, "sqrt", "log2"]),  
    "max_leaf_nodes": trial.suggest_int("max_leaf_nodes", 2, 1000),  
    "min_impurity_decrease": trial.suggest_float("min_impurity_decrease", 0.0, 1.0),  
    "ccp_alpha": trial.suggest_float("ccp_alpha", 0.0, 0.1),  
}
```

Results :

```
params = {  
    "criterion": "friedman_mse",  
    "splitter": "random",  
    "max_depth": 40,  
    "min_samples_split": 5,  
    "min_samples_leaf": 15,  
    "min_weight_fraction_leaf": 0.0003194808485743293,  
    "max_features": None,  
    "max_leaf_nodes": 886,  
    "min_impurity_decrease": 0.4983751874061905,  
    "ccp_alpha": 0.0062804080152011445  
}
```

- **R²** : 0.1957
- **MAE** : 0.5746
- **RMSE** : 0.7482



Linear Regression (classical) Since only 4 cases are possible, only 4 trials are ran.

```
params = {
    "copy_X": True,
    "fit_intercept": trial.suggest_categorical("fit_intercept", [True, False]),
    "positive": trial.suggest_categorical("positive", [True, False]),
}
```

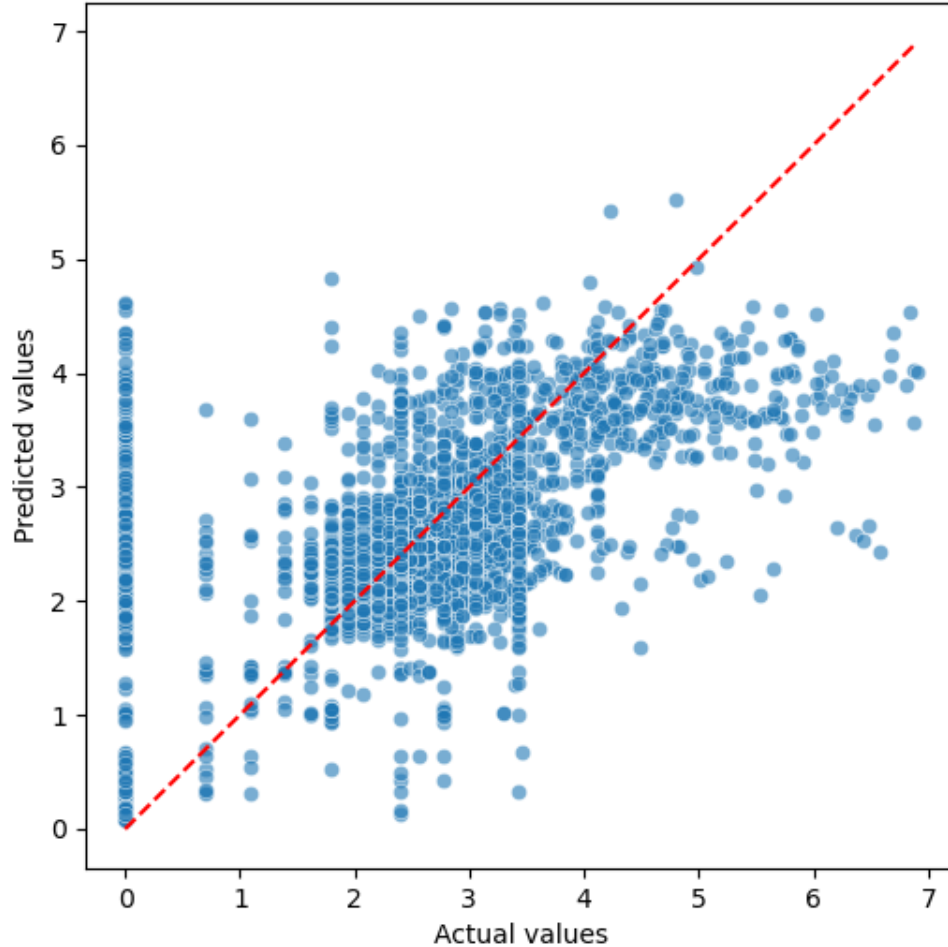
Results :

```
params = {
    "fit_intercept": False,
    "positive": True
}
```

- R^2 : 0.2853
- MAE : 0.5017

- RMSE : 0.6648

Linear Regression (classical) - Predicted vs Actual Delay ($R^2 = 0.285$)



Linear Regression (Elasticnet)

```
params = {
    "alpha": trial.suggest_float("alpha", 1e-6, 10.0, log=True),
    "l1_ratio": trial.suggest_float("l1_ratio", 0.0, 1.0),
    "fit_intercept": trial.suggest_categorical("fit_intercept", [True, False]),
    "max_iter": trial.suggest_int("max_iter", 500, 5000),
    "tol": trial.suggest_float("tol", 1e-6, 1e-2, log=True),
    "selection": trial.suggest_categorical("selection", ["cyclic", "random"])
}
```

Results :

```
params = {
    "alpha": 0.00038542540083133113,
```

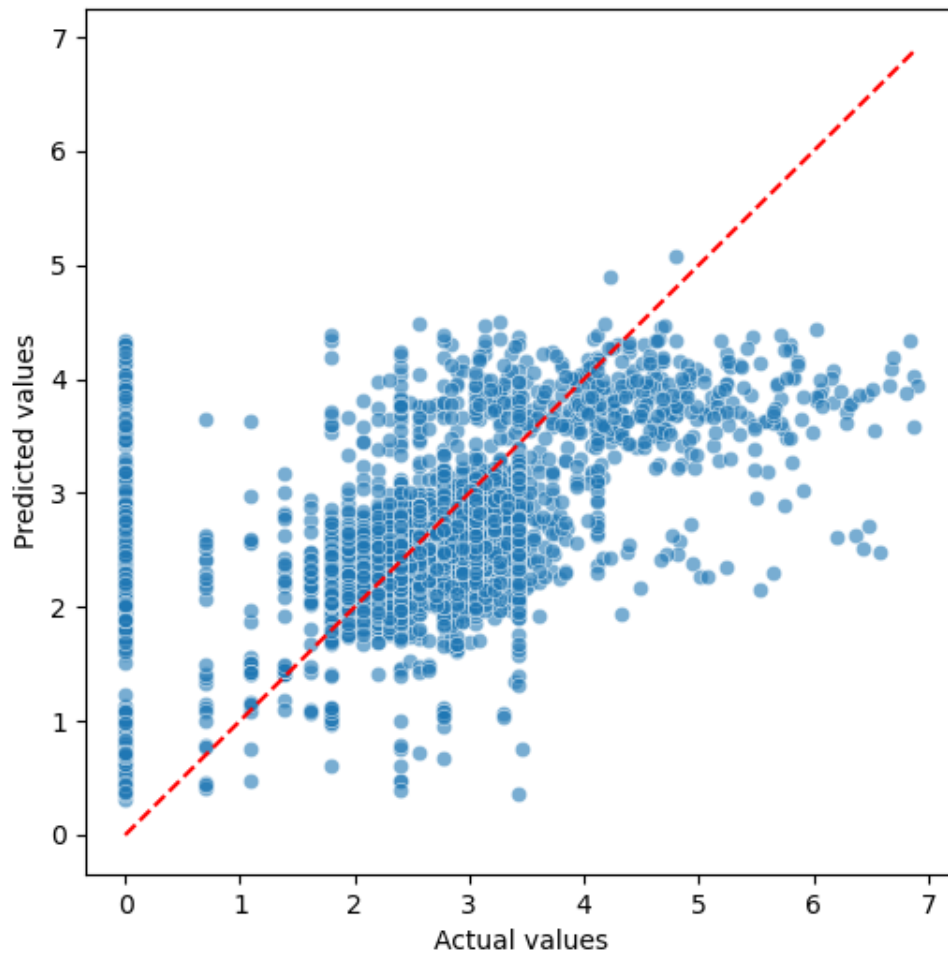
```

"l1_ratio": 0.11733064861417629,
"fit_intercept": True,
"max_iter": 1227,
"tol": 3.074079260630123e-05,
"selection": "random"
}

```

- R^2 : 0.2877
- MAE : 0.5043
- RMSE : 0.6626

Linear Regression (Elasticnet) - Predicted vs Actual Delay ($R^2 = 0.288$)



Random Forest We did not execute 100 trials due to extremely long execution times. We executed only 1 test with the following parameters.

Parameters :

```
params = {
```

```

    "n_estimators": trial.suggest_int("n_estimators", 50, 600),
    "criterion": trial.suggest_categorical(
        "criterion", ["squared_error", "absolute_error", "friedman_mse", "poisson"]
    ),
    "max_depth": trial.suggest_int("max_depth", 2, 40),
    "min_samples_split": trial.suggest_int("min_samples_split", 2, 20),
    "min_samples_leaf": trial.suggest_int("min_samples_leaf", 1, 20),
    "min_weight_fraction_leaf": trial.suggest_float("min_weight_fraction_leaf", 0.0, 0.5),
    "max_features": trial.suggest_categorical(
        "max_features", ["sqrt", "log2", None]
    ),
    "max_leaf_nodes": trial.suggest_int("max_leaf_nodes", 100, 5000),
    "min_impurity_decrease": trial.suggest_float("min_impurity_decrease", 0.0, 1.0),
    "bootstrap": trial.suggest_categorical("bootstrap", [True, False]),
    "oob_score": trial.suggest_categorical("oob_score", [False, True]),
    "ccp_alpha": trial.suggest_float("ccp_alpha", 0.0, 0.1)
}

```

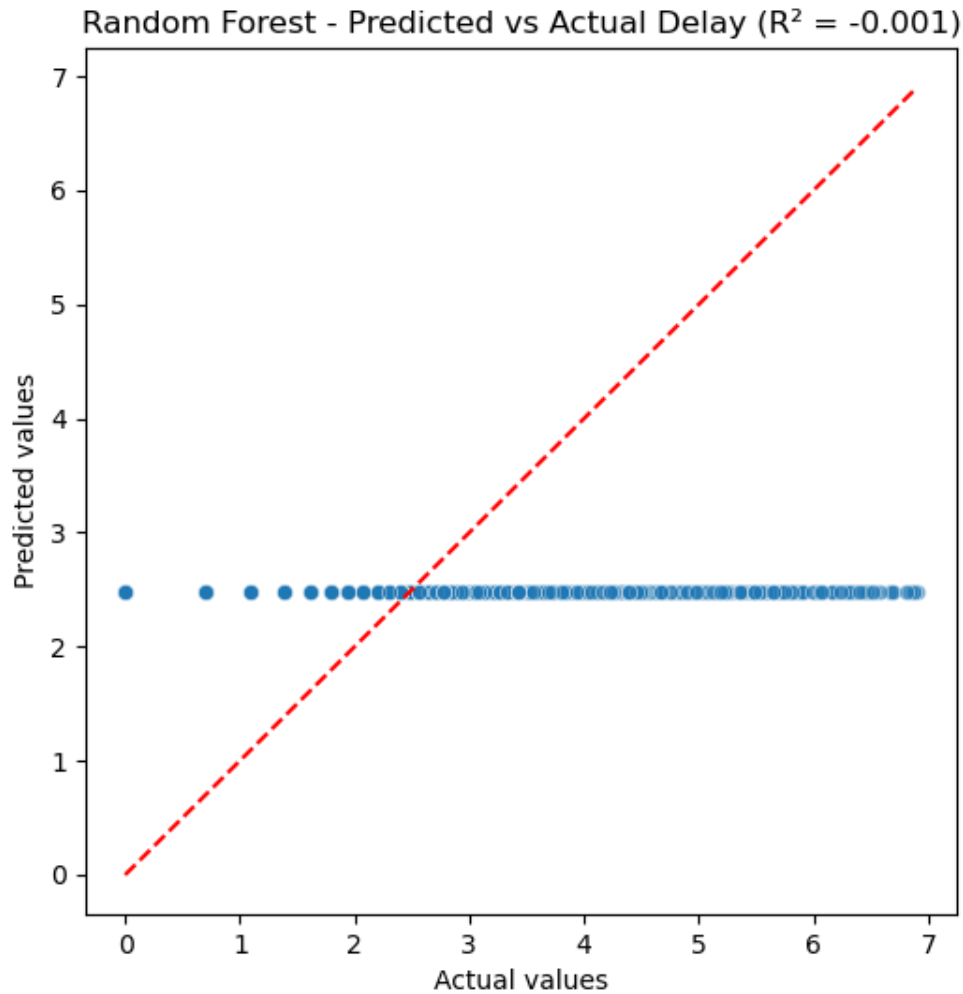
Results :

```

params = {
    "n_estimators": 95,
    "criterion": "absolute_error",
    "max_depth": 25,
    "min_samples_split": 12,
    "min_samples_leaf": 2,
    "min_weight_fraction_leaf": 0.45092598137957596,
    "max_features": None,
    "max_leaf_nodes": 1589,
    "min_impurity_decrease": 0.16578563594061557,
    "bootstrap": False,
    "oob_score": False,
    "ccp_alpha": 0.02901108238619291
}

```

- **R²** : -0.0009
- **MAE** : 0.6217
- **RMSE** : 0.9310



Gradient Boosting Parameters :

```
params = {
    "loss": trial.suggest_categorical(
        "loss", ["squared_error", "absolute_error", "huber", "quantile"]
    ),
    "learning_rate": trial.suggest_float("learning_rate", 0.001, 0.3, log=True),
    "n_estimators": trial.suggest_int("n_estimators", 50, 800),
    "subsample": trial.suggest_float("subsample", 0.5, 1.0),
    "criterion": trial.suggest_categorical(
        "criterion", ["friedman_mse", "squared_error"]
    ),
    "min_samples_split": trial.suggest_int("min_samples_split", 2, 20),
    "min_samples_leaf": trial.suggest_int("min_samples_leaf", 1, 20),
    "min_weight_fraction_leaf": trial.suggest_float("min_weight_fraction_leaf", 0.0, 0.3),
    "max_depth": trial.suggest_int("max_depth", 2, 10),
}
```

```

    "max_features": trial.suggest_categorical(
        "max_features", ["sqrt", "log2", None]
    ),
    "min_impurity_decrease": trial.suggest_float("min_impurity_decrease", 0.0, 1.0),
    "alpha": trial.suggest_float("alpha", 0.01, 0.99), # used for quantile/huber losses
}

```

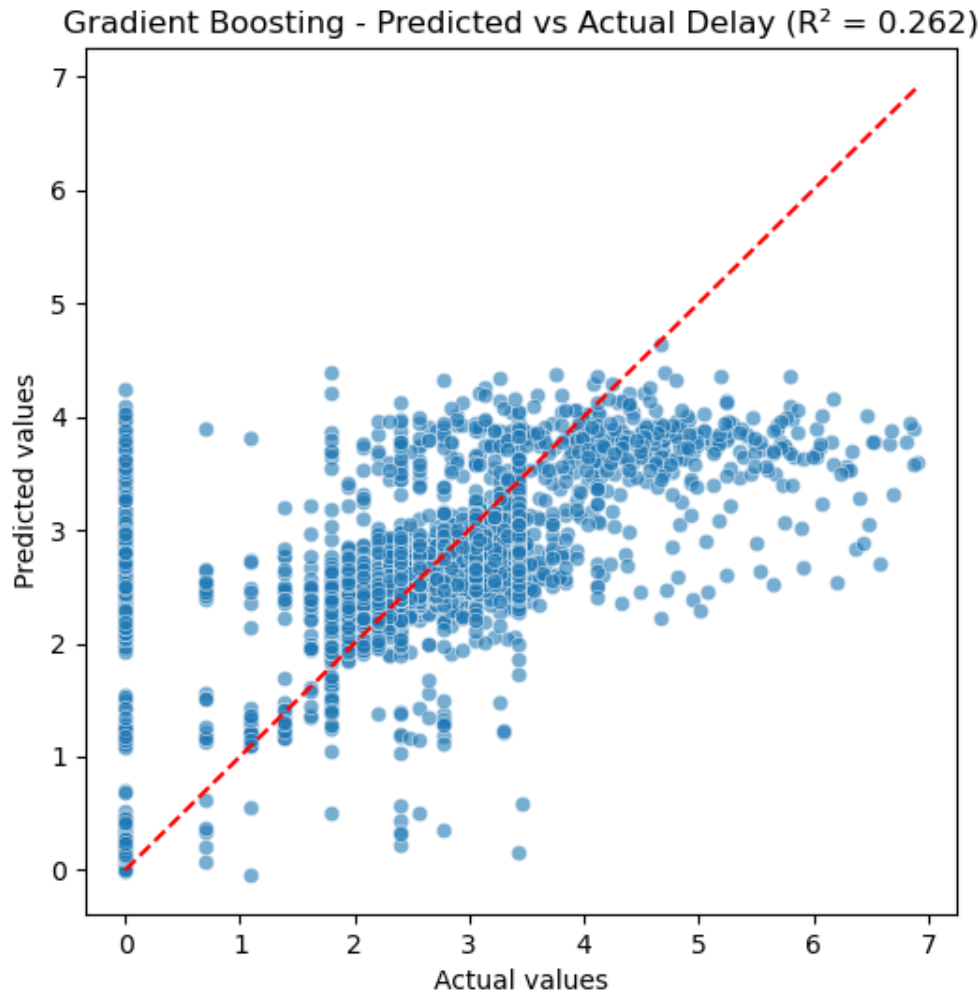
Results :

```

params = {
    "loss": "absolute_error"
    "learning_rate": 0.02280176533792217
    "n_estimators": 719
    "subsample": 0.7972495351158401
    "criterion": "friedman_mse"
    "min_samples_split": 20
    "min_samples_leaf": 3
    "min_weight_fraction_leaf": 0.0002052743467058696
    "max_depth": 9
    "max_features": "sqrt"
    "min_impurity_decrease": 0.20369418638457593
}

```

- R^2 : 0.2625
- MAE : 0.4650
- RMSE : 0.6860



Extreme Gradient Boosting Parameters :

```
params = {  
    "n_estimators": trial.suggest_int("n_estimators", 200, 1500),  
    "learning_rate": trial.suggest_float("learning_rate", 1e-4, 0.3, log=True),  
    # Tree parameters  
    "max_depth": trial.suggest_int("max_depth", 2, 12),  
    "min_child_weight": trial.suggest_float("min_child_weight", 1e-3, 10.0, log=True),  
    "gamma": trial.suggest_float("gamma", 0.0, 10.0),  
    "max_delta_step": trial.suggest_int("max_delta_step", 0, 10),  
    # Regularization  
    "reg_alpha": trial.suggest_float("reg_alpha", 1e-8, 10.0, log=True),  
    "reg_beta": trial.suggest_float("reg_lambda", 1e-8, 10.0, log=True),  
    # Subsampling  
    "subsample": trial.suggest_float("subsample", 0.5, 1.0),  
    "colsample_bytree": trial.suggest_float("colsample_bytree", 0.5, 1.0),  
}
```

```

"colsample_bylevel": trial.suggest_float("colsample_bylevel", 0.5, 1.0),
"colsample_bynode": trial.suggest_float("colsample_bynode", 0.5, 1.0),
# Booster & misc.
"booster": trial.suggest_categorical("booster", ["gbtree"]),
"tree_method": "auto",
"random_state": 99,
"n_jobs": -1,
}

```

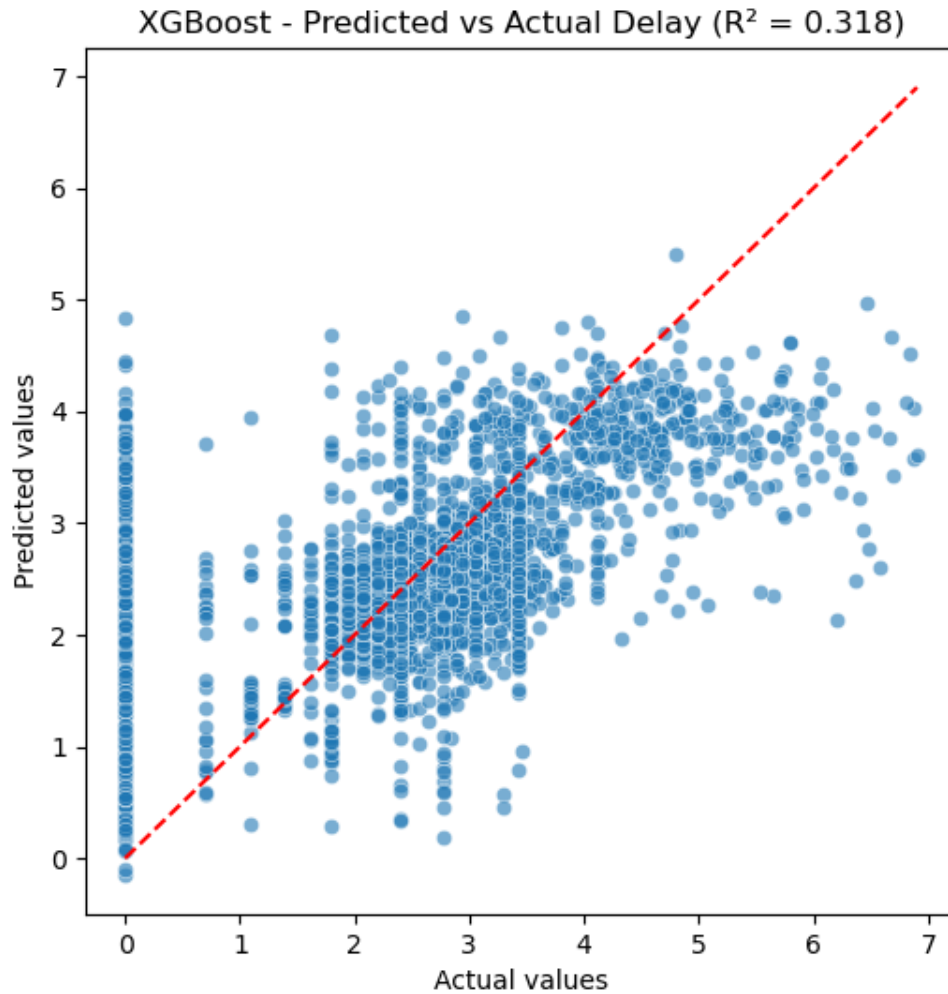
Results :

```

params = {
    "n_estimators": 806
    "learning_rate": 0.03029302502803171
    "max_depth": 8
    "min_child_weight": 0.0038121797269087977
    "gamma": 0.965193111361264
    "max_delta_step": 7
    "reg_alpha": 0.0034736346194687985
    "reg_lambda": 6.023943153925873
    "subsample": 0.7307550264958214
    "colsample_bytree": 0.5900537660484718
    "colsample_bylevel": 0.5196737746952808
    "colsample_bynode": 0.6219074422247004
    "booster": "gbtree"
}

```

- **R²** : 0.3182
- **MAE** : 0.4916
- **RMSE** : 0.6342



1.2.3 Support Vector Regression (SVR)

We did not execute 100 trials due to extremely long execution times. We executed only 5 tests with the following parameters.

Parameters :

```
kernel = trial.suggest_categorical("kernel", ["rbf", "poly", "sigmoid", "linear"])
params = {
    "kernel": kernel,
    "C": trial.suggest_float("C", 1e-3, 1e3, log=True),
    "epsilon": trial.suggest_float("epsilon", 1e-4, 1.0, log=True),
    "shrinking": trial.suggest_categorical("shrinking", [True, False]),
    "gamma": trial.suggest_categorical("gamma", ["scale", "auto"]),
}
# Kernel-specific parameters
```

```

if kernel == "poly":
    params["degree"] = trial.suggest_int("degree", 2, 5)
    params["coef0"] = trial.suggest_float("coef0", 0.0, 1.0)
elif kernel == "sigmoid":
    params["coef0"] = trial.suggest_float("coef0", 0.0, 1.0)

```

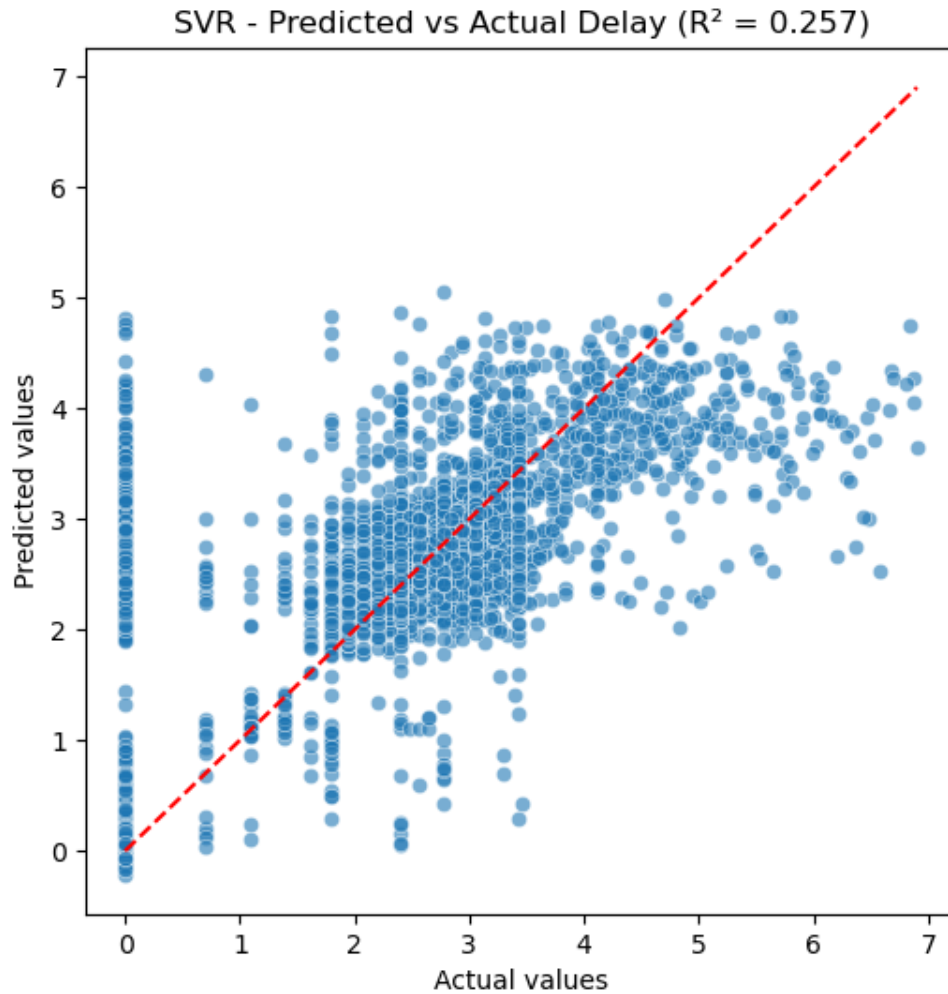
Results :

```

params = {
    "kernel": "poly"
    "C": 4.944955595207548
    "epsilon": 0.02175537301173423
    "shrinking": True
    "gamma": "scale"
    "degree": 2
    "coef0": 0.4962961764689876
}

```

- R^2 : 0.2570
- MAE : 0.4472
- RMSE : 0.6911



Scenario 3 - Classical models, Summer

- **Entry data :** 4_preprocessed_dataset_summer.csv
- **Script :** train_models_summer.py
- **Train/Test/Validation split :** Train (60%)/Validation (20%)/Test (20%)

Decision Tree Parameters :

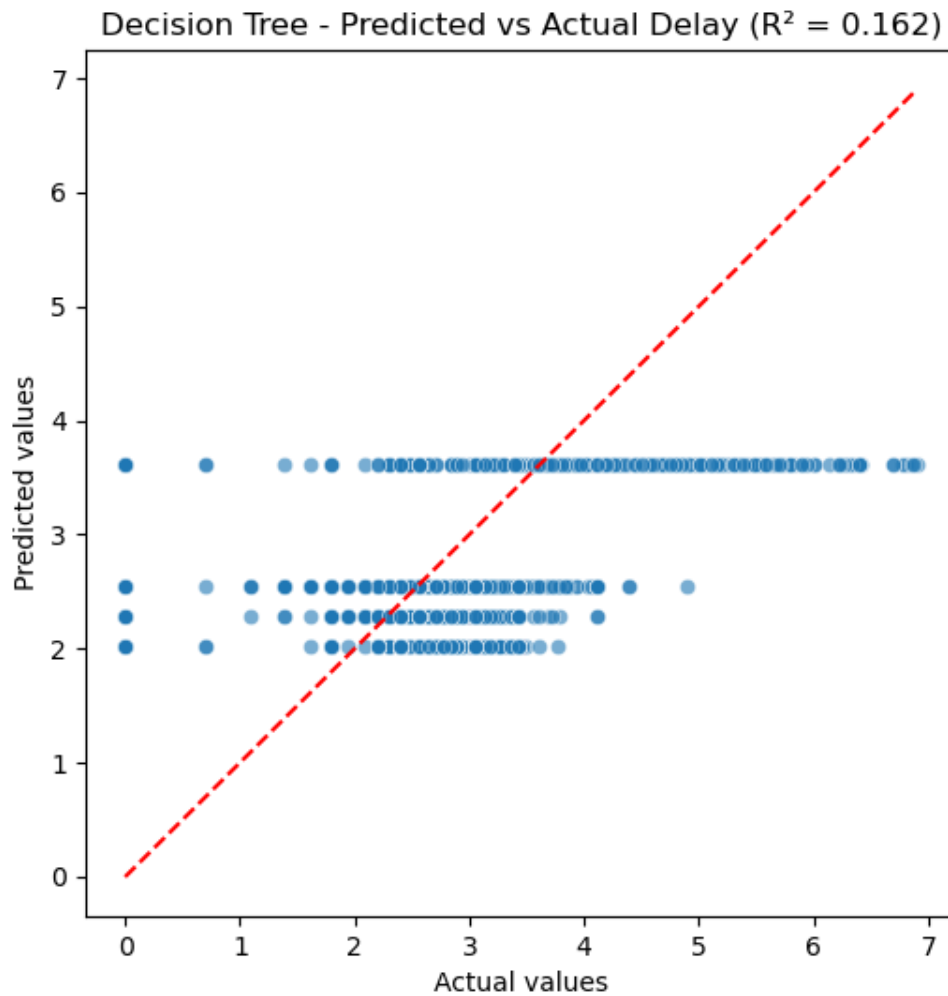
```
params = {
    "criterion": trial.suggest_categorical("criterion", ["squared_error", "friedman_mse", "abs",
    "splitter": trial.suggest_categorical("splitter", ["best", "random"]),
    "max_depth": trial.suggest_int("max_depth", 1, 50),
    "min_samples_split": trial.suggest_int("min_samples_split", 2, 20),
    "min_samples_leaf": trial.suggest_int("min_samples_leaf", 1, 20),
    "min_weight_fraction_leaf": trial.suggest_float("min_weight_fraction_leaf", 0.0, 0.5),
    "max_features": trial.suggest_categorical("max_features", [None, "sqrt", "log2"]),
```

```
"max_leaf_nodes": trial.suggest_int("max_leaf_nodes", 2, 1000),
"min_impurity_decrease": trial.suggest_float("min_impurity_decrease", 0.0, 1.0),
"ccp_alpha": trial.suggest_float("ccp_alpha", 0.0, 0.1),
}
```

Results :

```
params = {
    "criterion": "friedman_mse"
    "splitter": "best"
    "max_depth": 4
    "min_samples_split": 4
    "min_samples_leaf": 5
    "min_weight_fraction_leaf": 0.06298245386908075
    "max_features": None
    "max_leaf_nodes": 870
    "min_impurity_decrease": 0.3937352951676524
}
```

- **R²** : 0.1615
- **MAE** : 0.5799
- **RMSE** : 0.7578



Linear Regression (classical) Since only 4 cases are possible, only 4 trials are ran.

```
params = {
    "copy_X": True,
    "fit_intercept": trial.suggest_categorical("fit_intercept", [True, False]),
    "positive": trial.suggest_categorical("positive", [True, False]),
}
```

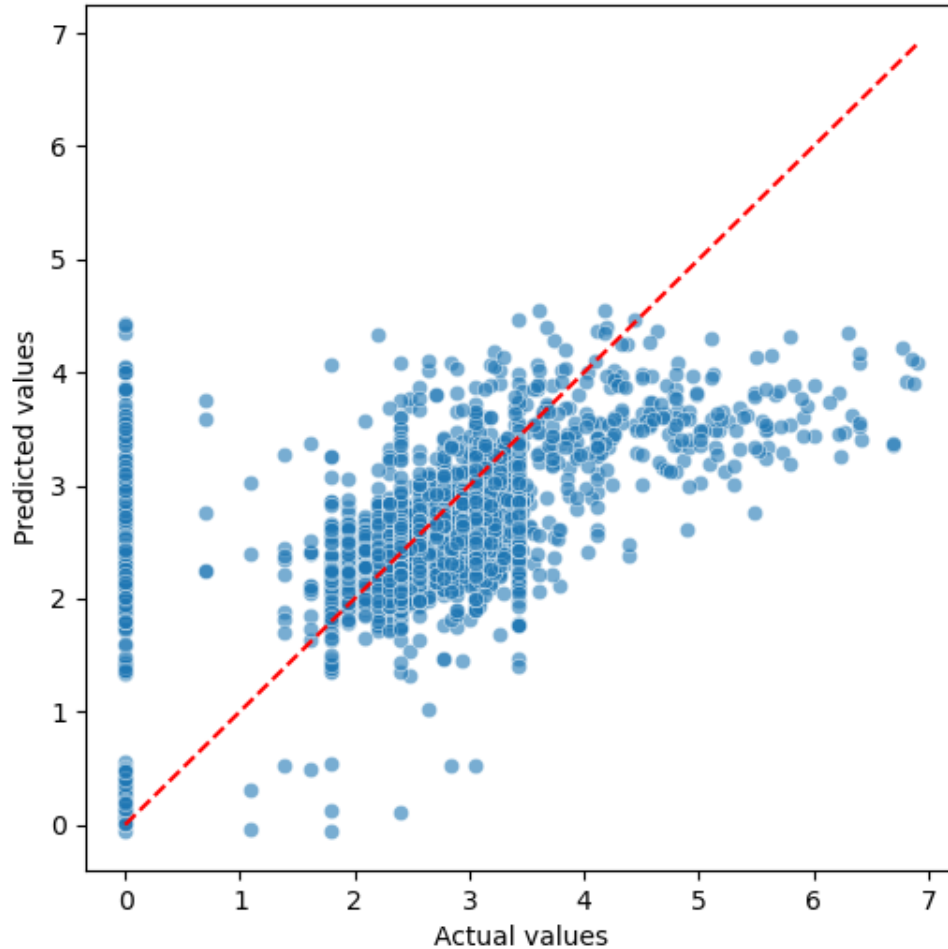
Results :

```
params = {
    "fit_intercept": False,
    "positive": True
}
```

- R^2 : 0.2826
- MAE : 0.4824

- RMSE : 0.6484

Linear Regression (classical) - Predicted vs Actual Delay ($R^2 = 0.283$)



Linear Regression (Elasticnet)

```
params = {
    "alpha": trial.suggest_float("alpha", 1e-6, 10.0, log=True),
    "l1_ratio": trial.suggest_float("l1_ratio", 0.0, 1.0),
    "fit_intercept": trial.suggest_categorical("fit_intercept", [True, False]),
    "max_iter": trial.suggest_int("max_iter", 500, 5000),
    "tol": trial.suggest_float("tol", 1e-6, 1e-2, log=True),
    "selection": trial.suggest_categorical("selection", ["cyclic", "random"])
}
```

Results :

```
params = {
    "alpha": 0.0005415904606672672,
```

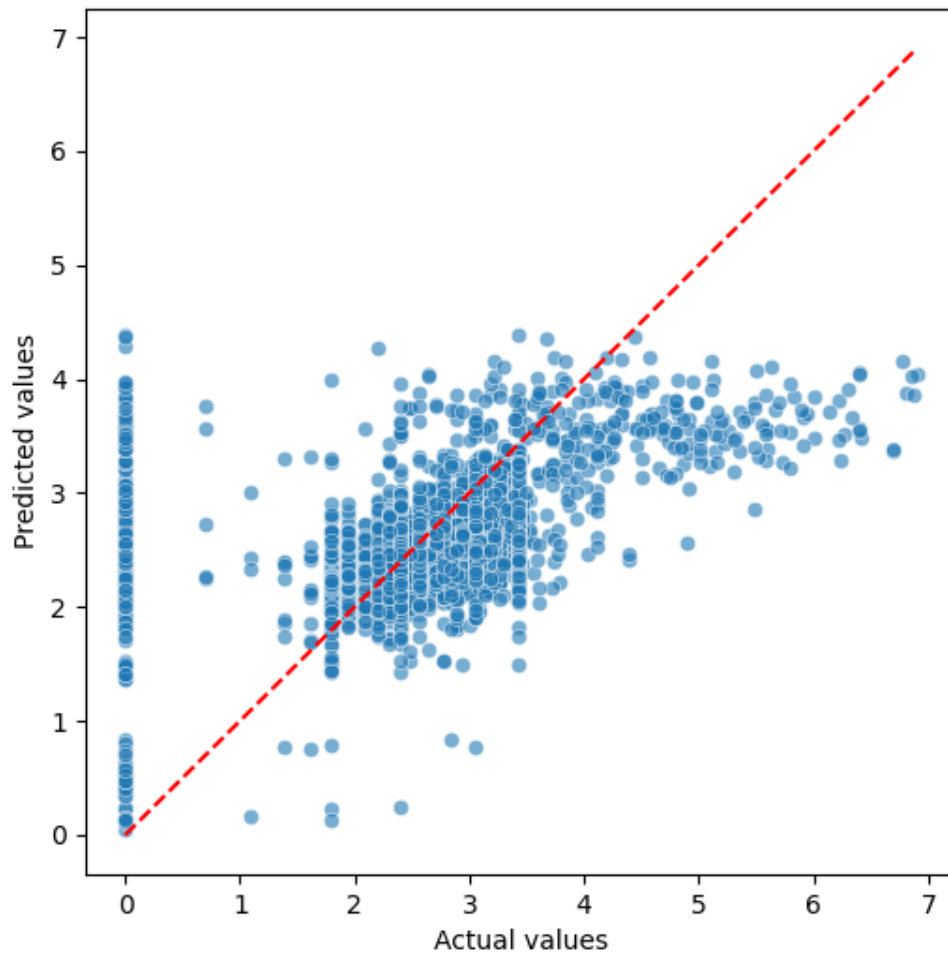
```

    "l1_ratio": 0.06711143766800608,
    "fit_intercept": True,
    "max_iter": 1031,
    "tol": 0.0025125336203729955,
    "selection": "cyclic"
}

```

- R^2 : 0.2865
- MAE : 0.4874
- RMSE : 0.6449

Linear Regression (Elasticnet) - Predicted vs Actual Delay ($R^2 = 0.286$)



Random Forest We did not execute 100 trials due to extremely long execution times. We executed only 5 tests with the following parameters.

Parameters :

```
params = {
```

```

    "n_estimators": trial.suggest_int("n_estimators", 50, 600),
    "criterion": trial.suggest_categorical(
        "criterion", ["squared_error", "absolute_error", "friedman_mse", "poisson"]
    ),
    "max_depth": trial.suggest_int("max_depth", 2, 40),
    "min_samples_split": trial.suggest_int("min_samples_split", 2, 20),
    "min_samples_leaf": trial.suggest_int("min_samples_leaf", 1, 20),
    "min_weight_fraction_leaf": trial.suggest_float("min_weight_fraction_leaf", 0.0, 0.5),
    "max_features": trial.suggest_categorical(
        "max_features", ["sqrt", "log2", None]
    ),
    "max_leaf_nodes": trial.suggest_int("max_leaf_nodes", 100, 5000),
    "min_impurity_decrease": trial.suggest_float("min_impurity_decrease", 0.0, 1.0),
    "bootstrap": trial.suggest_categorical("bootstrap", [True, False]),
    "oob_score": trial.suggest_categorical("oob_score", [False, True]),
    "ccp_alpha": trial.suggest_float("ccp_alpha", 0.0, 0.1)
}

```

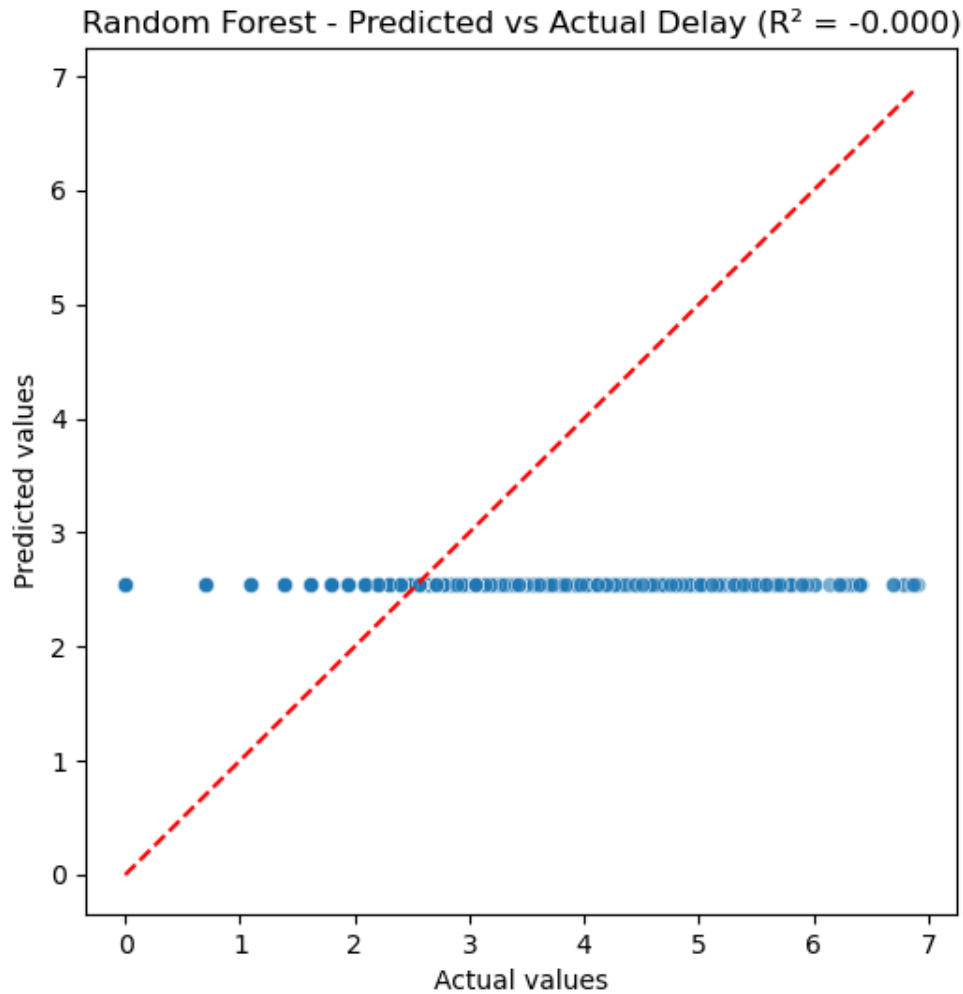
Results :

```

params = {
    "n_estimators": 339,
    "criterion": "friedman_mse",
    "max_depth": 7,
    "min_samples_split": 9,
    "min_samples_leaf": 17,
    "min_weight_fraction_leaf": 0.4678768997063537,
    "max_features": "sqrt",
    "max_leaf_nodes": 952,
    "min_impurity_decrease": 0.3854587350914097,
    "bootstrap": False,
    "oob_score": False,
    "ccp_alpha": 0.0862227243102725
}

```

- **R²** : -0.0002
- **MAE** : 0.6112
- **RMSE** : 0.9040



Gradient Boosting Parameters :

```
params = {
    "loss": trial.suggest_categorical(
        "loss", ["squared_error", "absolute_error", "huber", "quantile"]
    ),
    "learning_rate": trial.suggest_float("learning_rate", 0.001, 0.3, log=True),
    "n_estimators": trial.suggest_int("n_estimators", 50, 800),
    "subsample": trial.suggest_float("subsample", 0.5, 1.0),
    "criterion": trial.suggest_categorical(
        "criterion", ["friedman_mse", "squared_error"]
    ),
    "min_samples_split": trial.suggest_int("min_samples_split", 2, 20),
    "min_samples_leaf": trial.suggest_int("min_samples_leaf", 1, 20),
    "min_weight_fraction_leaf": trial.suggest_float("min_weight_fraction_leaf", 0.0, 0.3),
    "max_depth": trial.suggest_int("max_depth", 2, 10),
}
```

```

    "max_features": trial.suggest_categorical(
        "max_features", ["sqrt", "log2", None]
    ),
    "min_impurity_decrease": trial.suggest_float("min_impurity_decrease", 0.0, 1.0),
    "alpha": trial.suggest_float("alpha", 0.01, 0.99), # used for quantile/huber losses
}

```

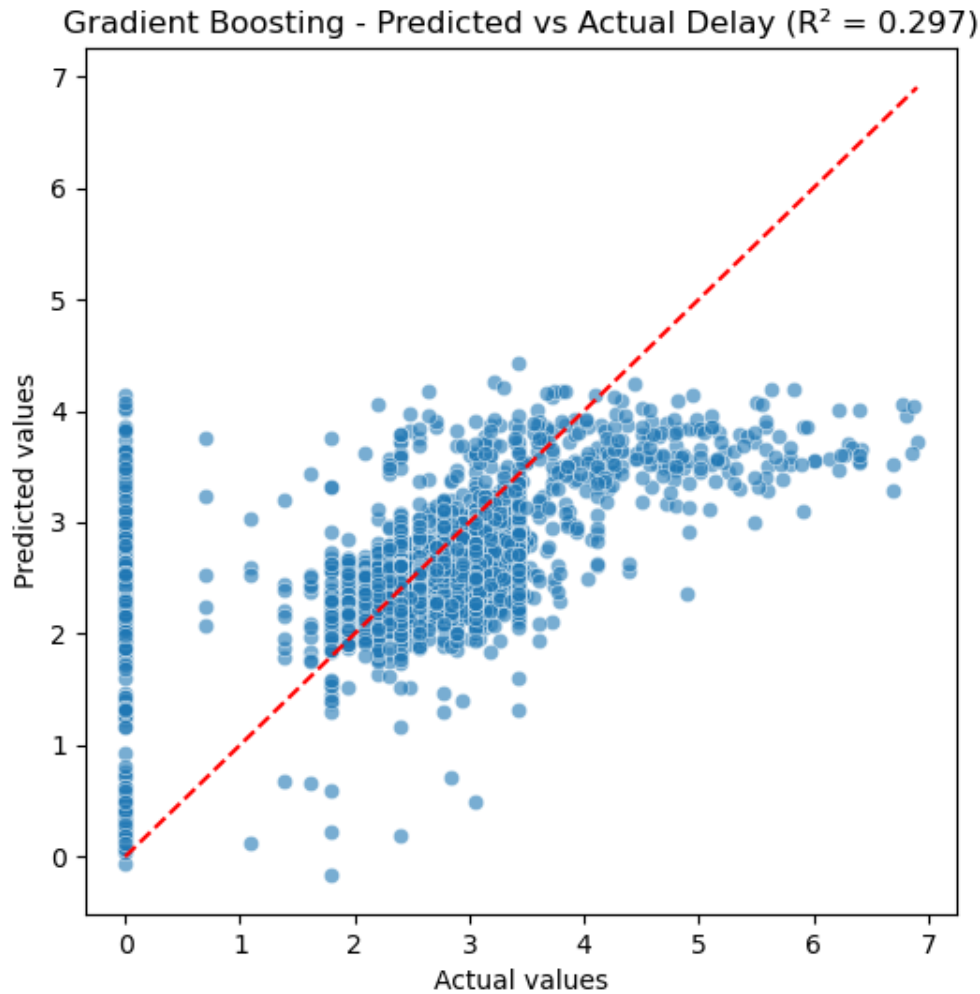
Results :

```

params = {
    "loss": "squared_error",
    "learning_rate": 0.08493400397168298,
    "n_estimators": 355,
    "subsample": 0.5097553445692088,
    "criterion": "friedman_mse",
    "min_samples_split": 4,
    "min_samples_leaf": 18,
    "min_weight_fraction_leaf": 0.0014236179282328368,
    "max_depth": 4,
    "max_features": "sqrt",
    "min_impurity_decrease": 0.8791889749525543,
    "alpha": 0.5602793761231738
}

```

- **R²** : 0.2969
- **MAE** : 0.4882
- **RMSE** : 0.6354



Extreme Gradient Boosting Parameters :

```
params = {
    "n_estimators": trial.suggest_int("n_estimators", 200, 1500),
    "learning_rate": trial.suggest_float("learning_rate", 1e-4, 0.3, log=True),
    # Tree parameters
    "max_depth": trial.suggest_int("max_depth", 2, 12),
    "min_child_weight": trial.suggest_float("min_child_weight", 1e-3, 10.0, log=True),
    "gamma": trial.suggest_float("gamma", 0.0, 10.0),
    "max_delta_step": trial.suggest_int("max_delta_step", 0, 10),
    # Regularization
    "reg_alpha": trial.suggest_float("reg_alpha", 1e-8, 10.0, log=True),
    "reg_beta": trial.suggest_float("reg_lambda", 1e-8, 10.0, log=True),
    # Subsampling
    "subsample": trial.suggest_float("subsample", 0.5, 1.0),
    "colsample_bytree": trial.suggest_float("colsample_bytree", 0.5, 1.0),
```

```

"colsample_bylevel": trial.suggest_float("colsample_bylevel", 0.5, 1.0),
"colsample_bynode": trial.suggest_float("colsample_bynode", 0.5, 1.0),
# Booster & misc.
"booster": trial.suggest_categorical("booster", ["gbtree"]),
"tree_method": "auto",
"random_state": 99,
"n_jobs": -1,
}

```

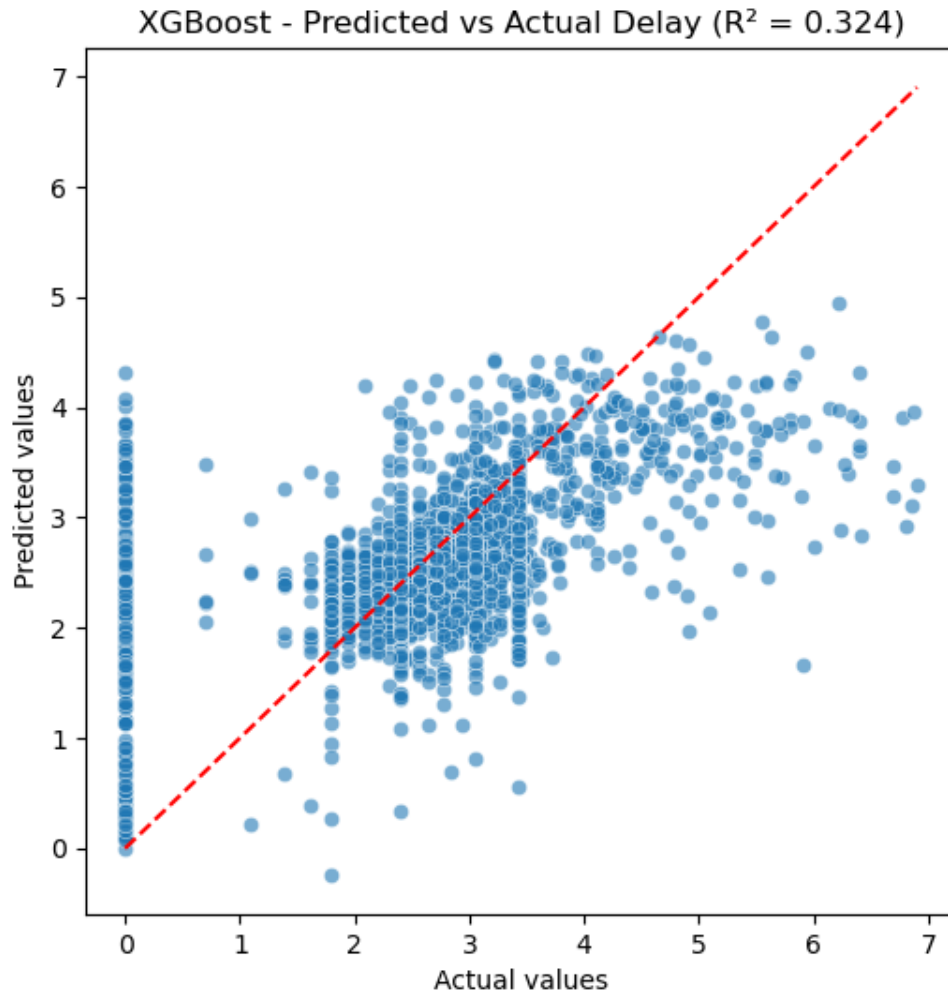
Results :

```

params = {
    "n_estimators": 1494,
    "learning_rate": 0.0170096189751765,
    "max_depth": 9,
    "min_child_weight": 5.340376980313727,
    "gamma": 0.5052289826389069,
    "max_delta_step": 5,
    "reg_alpha": 0.007644430950964261,
    "reg_lambda": 9.919662875022942,
    "subsample": 0.8398912127397786,
    "colsample_bytree": 0.5588507034087213,
    "colsample_bylevel": 0.8167534931699498,
    "colsample_bynode": 0.704605503225567,
    "booster": "gbtree"
}

```

- **R²** : 0.3245
- **MAE** : 0.4712
- **RMSE** : 0.6106



Support Vector Regression (SVR) We did not execute 100 trials due to extremely long execution times. We executed only 5 tests with the following parameters.

Parameters :

```
kernel = trial.suggest_categorical("kernel", ["rbf", "poly", "sigmoid", "linear"])
params = {
    "kernel": kernel,
    "C": trial.suggest_float("C", 1e-3, 1e3, log=True),
    "epsilon": trial.suggest_float("epsilon", 1e-4, 1.0, log=True),
    "shrinking": trial.suggest_categorical("shrinking", [True, False]),
    "gamma": trial.suggest_categorical("gamma", ["scale", "auto"]),
}
# Kernel-specific parameters
if kernel == "poly":
    params["degree"] = trial.suggest_int("degree", 2, 5)
```

```

    params["coef0"] = trial.suggest_float("coef0", 0.0, 1.0)
elif kernel == "sigmoid":
    params["coef0"] = trial.suggest_float("coef0", 0.0, 1.0)

```

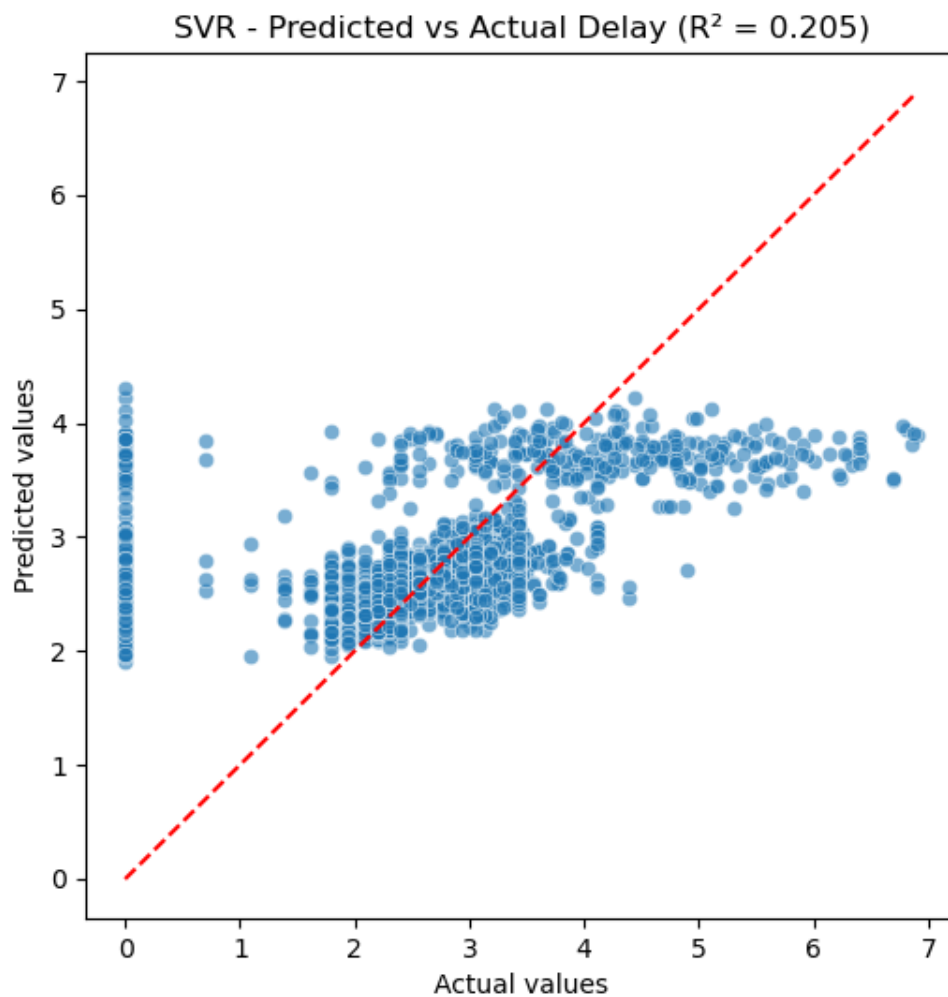
Results :

```

params = {
    "kernel": "rbf",
    "C": 1.0202504419935043,
    "epsilon": 0.17989837469900855,
    "shrinking": True,
    "gamma": "auto"
}

```

- R^2 : 0.2053
- MAE : 0.4885
- RMSE : 0.7183



Scenario 4 - GAMs, All seasons

- **Entry data** : 4_preprocessed_dataset_pygam.csv
- **Script** : train_pygam.py
- **Train/Test/Validation split** : Train (60%)/Validation (20%)/Test (20%)

With the following data specifications :

```
generic_cols_transf = (  
    f(0) + # ROUTE - nominal  
    f(1) + # INCIDENT - nominal  
    f(2) + # SEASON - nominal  
    s(3) + # VISIBILITY - ordinal  
    f(4) + # Clear - binary  
    f(5) + # Fog - binary  
    f(6) + # Rain - binary  
    f(7) + # Snow - binary  
    f(8) + # Thunderstorms - binary  
    s(9, basis="cp", edge_knots=[0, 24]) + # LOCAL_TIME_HOUR - cyclical  
    s(10, basis="cp", edge_knots=[0, 60]) + # LOCAL_TIME_MINUTE - cyclical  
    s(11, basis="cp", edge_knots=[0, 7]) + # WEEK_DAY - cyclical  
    s(12, basis="cp", edge_knots=[1, 12]) + # LOCAL_MONTH - cyclical  
    s(13, basis="cp", edge_knots=[1, 31]) + # LOCAL_DAY - cyclical  
    s(14, basis="cp", edge_knots=[0, 360]) + # WIND_DIRECTION - cyclical  
    f(15) + # PRECIP_AMOUNT_BINARY - binary  
    s(16) + # TEMP - continuous  
    s(17) + # DEW_POINT_TEMP - continuous  
    s(18) + # HUMIDEX - continuous  
    s(19) + # RELATIVE_HUMIDITY - continuous  
    s(20) + # STATION_PRESSURE - continuous  
    s(21) + # WIND_SPEED - continuous  
)
```

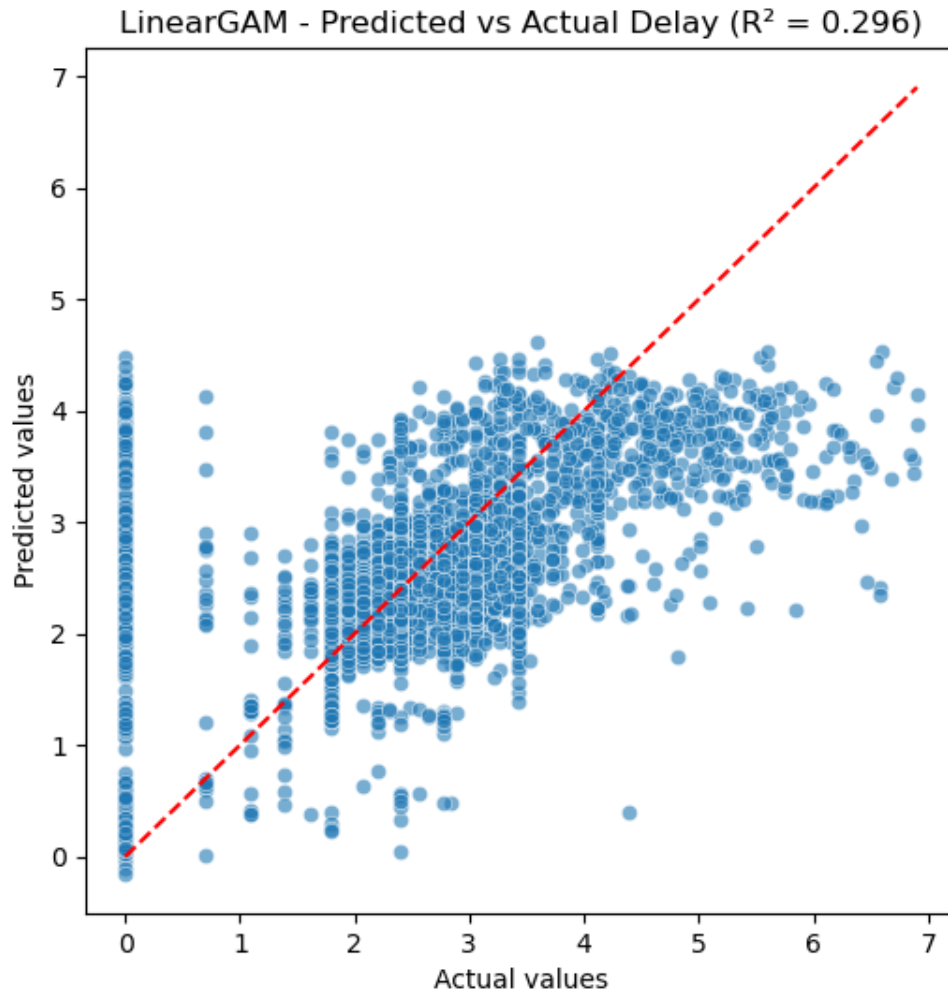
LinearGAM Parameters :

```
params = {  
    "n_splines": n_splines,  
    "lam": trial.suggest_float("lam", 1e-6, 1e3, log=True),  
    "max_iter": trial.suggest_int("max_iter", 100, 1000),  
    "tol": trial.suggest_float("tol", 1e-6, 1e-3, log=True),  
}
```

Results :

```
params = {  
    "n_splines": 5,  
    "lam": 0.3879848363855412,  
    "max_iter": 949,  
    "tol": 1.040858968295515e-06  
}
```

- R^2 : 0.2958
- MAE : 0.4896
- RMSE : 0.6526



GammaGAM Parameters :

```
params = {
    "n_splines": trial.suggest_int("n_splines", 5, 30),
    "lam": trial.suggest_float("lam", 1e-6, 1e3, log=True),
    "max_iter": trial.suggest_int("max_iter", 100, 1000),
    "tol": trial.suggest_float("tol", 1e-6, 1e-3, log=True),
}
```

Results :

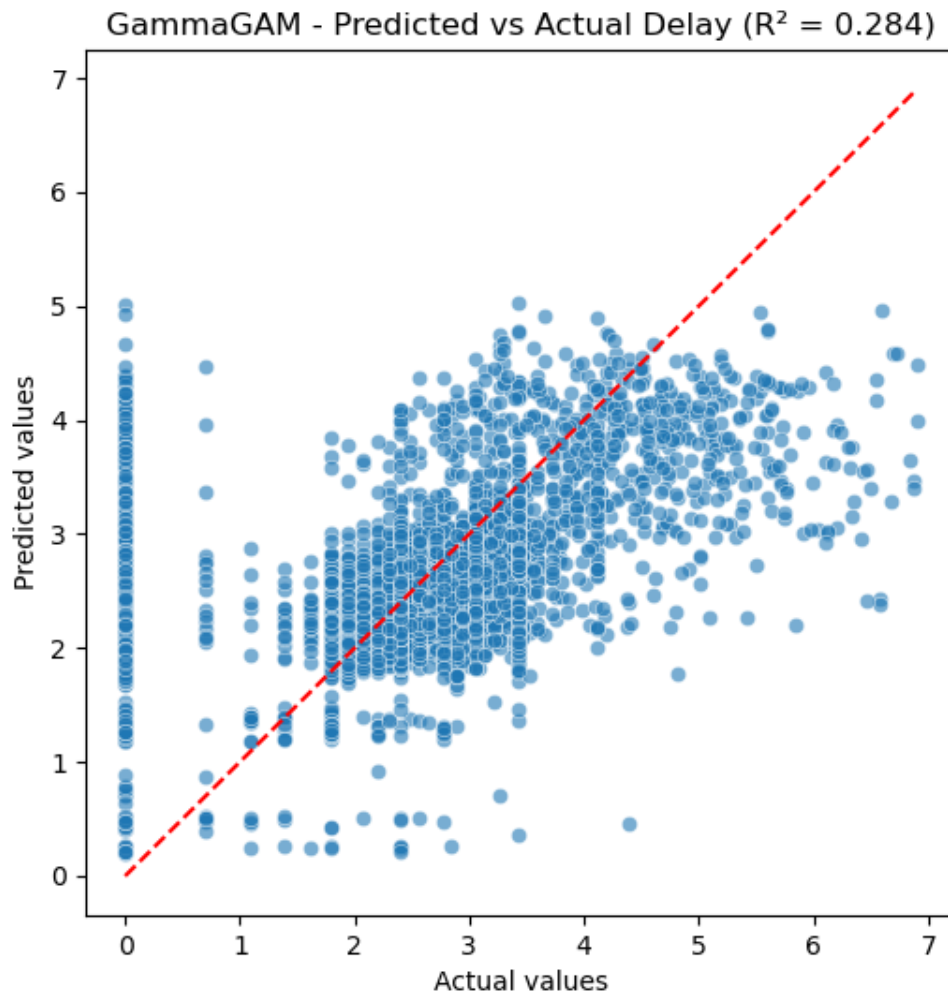
```
params = {
    "n_splines": 7,
```

```

    "lam": 6.996430731372631,
    "max_iter": 451,
    "tol": 2.2074610650990642e-05
}

```

- R^2 : 0.2838
- MAE : 0.4975
- RMSE : 0.6637



PoissonGAM We did not execute 100 trials due to long execution times. We executed only 25 tests with the following parameters.

Parameters :

```

params = {
    "n_splines": trial.suggest_int("n_splines", 5, 30),
    "lam": trial.suggest_float("lam", 1e-6, 1e3, log=True),
}

```

```

    "max_iter": trial.suggest_int("max_iter", 100, 1000),
    "tol": trial.suggest_float("tol", 1e-6, 1e-3, log=True),
}

```

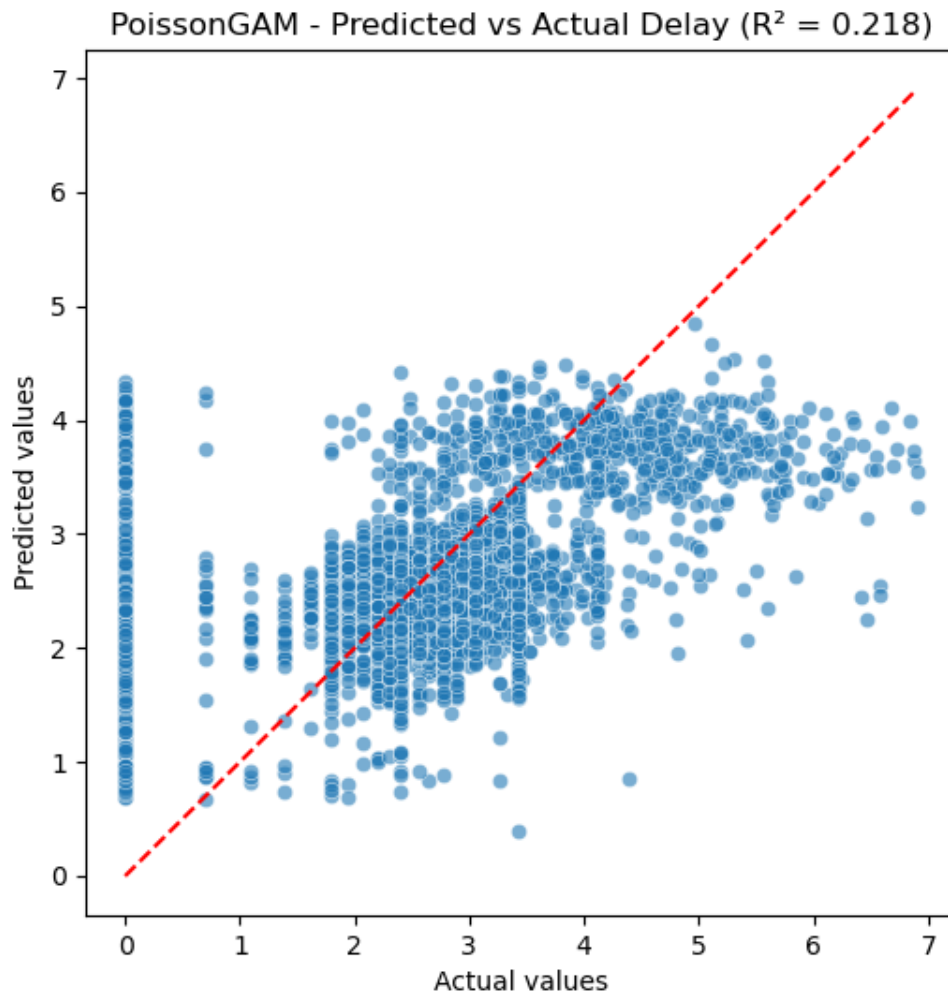
Results :

```

params = {
    "n_splines": 26,
    "lam": 1.9491937350065412e-05,
    "max_iter": 844,
    "tol": 4.3436155909000126e-06
}

```

- R^2 : 0.2179
- MAE : 0.5648
- RMSE : 0.7248



Scenario 5 - GAMs, Winter

- **Entry data** : 4_preprocessed_dataset_pygam_winter.csv
- **Script** : train_pygam_winter.py
- **Train/Test/Validation split** : Train (80%)/Validation (10%)/Test (10%)

With the following data specifications :

```
generic_cols_transf = (  
    f(0) + # ROUTE - nominal  
    f(1) + # INCIDENT - nominal  
    s(2) + # VISIBILITY - ordinal  
    f(3) + # Clear - binary  
    f(4) + # Fog - binary  
    f(5) + # Rain - binary  
    f(6) + # Snow - binary  
    f(7) + # Thunderstorms - binary  
    s(8, basis="cp", edge_knots=[0, 24]) + # LOCAL_TIME_HOUR - cyclical  
    s(9, basis="cp", edge_knots=[0, 60]) + # LOCAL_TIME_MINUTE - cyclical  
    s(10, basis="cp", edge_knots=[0, 7]) + # WEEK_DAY - cyclical  
    s(11, basis="cp", edge_knots=[1, 12]) + # LOCAL_MONTH - cyclical  
    s(12, basis="cp", edge_knots=[1, 31]) + # LOCAL_DAY - cyclical  
    s(13, basis="cp", edge_knots=[0, 360]) + # WIND_DIRECTION - cyclical  
    f(14) + # PRECIP_AMOUNT_BINARY - binary  
    s(15) + # TEMP - continuous  
    s(16) + # DEW_POINT_TEMP - continuous  
    s(17) + # HUMIDEX - continuous  
    s(18) + # RELATIVE_HUMIDITY - continuous  
    s(19) + # STATION_PRESSURE - continuous  
    s(20) + # WIND_SPEED - continuous  
)
```

LinearGAM Parameters :

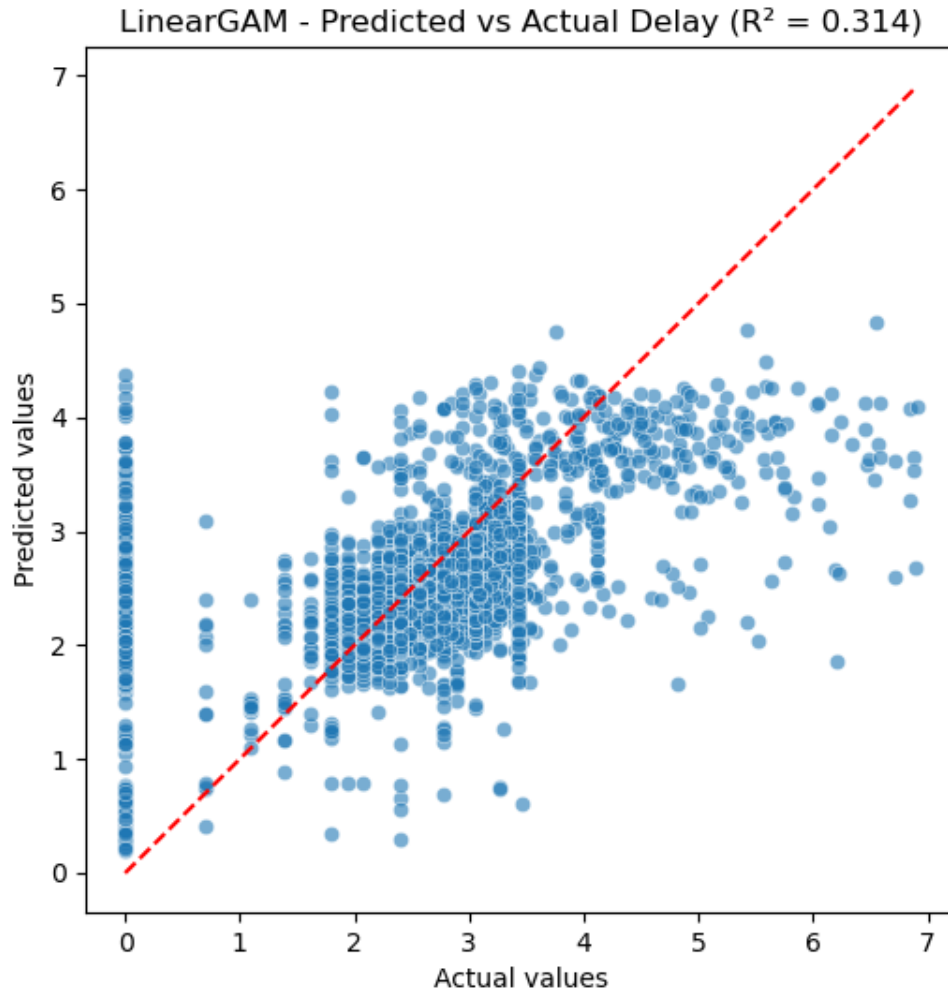
```
params = {  
    "n_splines": n_splines,  
    "lam": trial.suggest_float("lam", 1e-6, 1e3, log=True),  
    "max_iter": trial.suggest_int("max_iter", 100, 1000),  
    "tol": trial.suggest_float("tol", 1e-6, 1e-3, log=True),  
}
```

Results :

```
params = {  
    "n_splines": 5,  
    "lam": 6.400741604996376,  
    "max_iter": 500,  
    "tol": 0.00028849210409862127  
}
```

- R^2 : 0.3142

- MAE : 0.4947
- RMSE : 0.6515



GammaGAM Parameters :

```
params = {
    "n_splines": trial.suggest_int("n_splines", 5, 30),
    "lam": trial.suggest_float("lam", 1e-6, 1e3, log=True),
    "max_iter": trial.suggest_int("max_iter", 100, 1000),
    "tol": trial.suggest_float("tol", 1e-6, 1e-3, log=True),
}
```

Results :

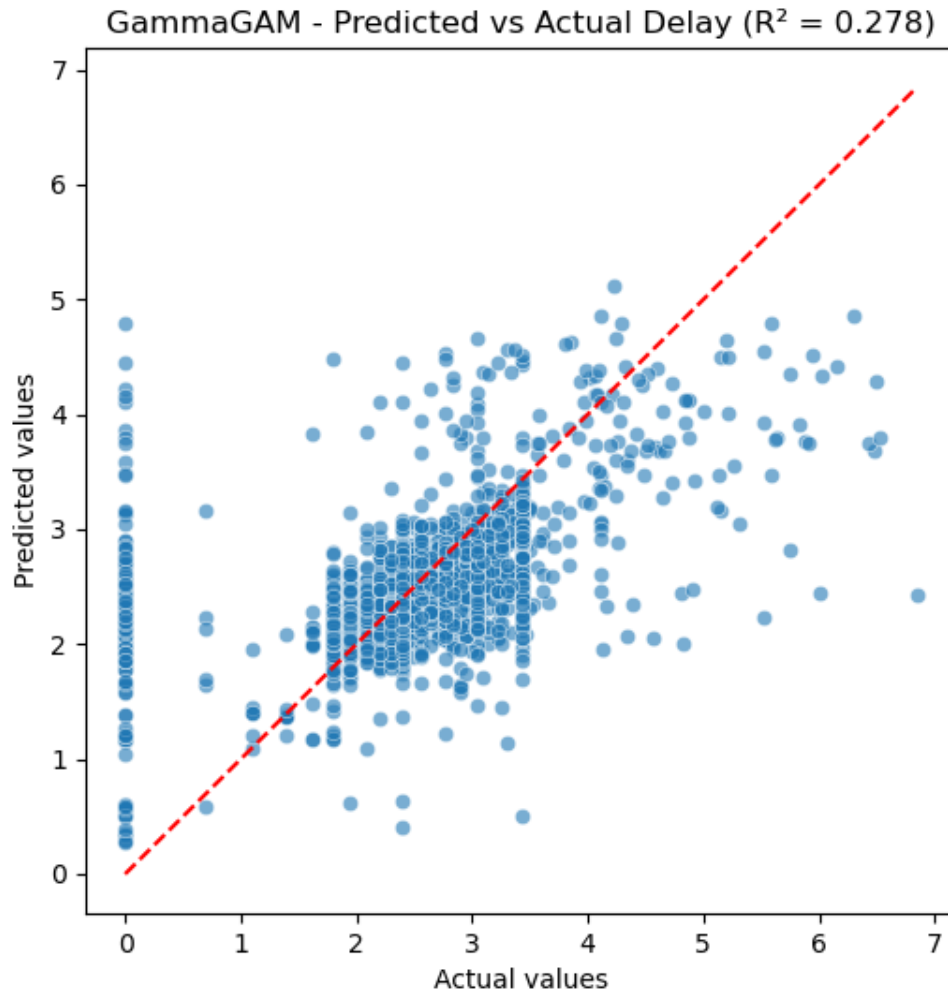
```
params = {
    "n_splines": 8,
    "lam": 4.137967988157594,
```

```

    "max_iter": 291,
    "tol": 9.0734547318602e-05
}

```

- **R2** : 0.2781
- **MAE** : 0.4929
- **RMSE** : 0.6372



PoissonGAM We did not execute 100 trials due to long execution times. We executed only 25 tests with the following parameters.

Parameters :

```

params = {
    "n_splines": trial.suggest_int("n_splines", 5, 30),
    "lam": trial.suggest_float("lam", 1e-6, 1e3, log=True),
    "max_iter": trial.suggest_int("max_iter", 100, 1000),
}

```

```

    "tol": trial.suggest_float("tol", 1e-6, 1e-3, log=True),
}

```

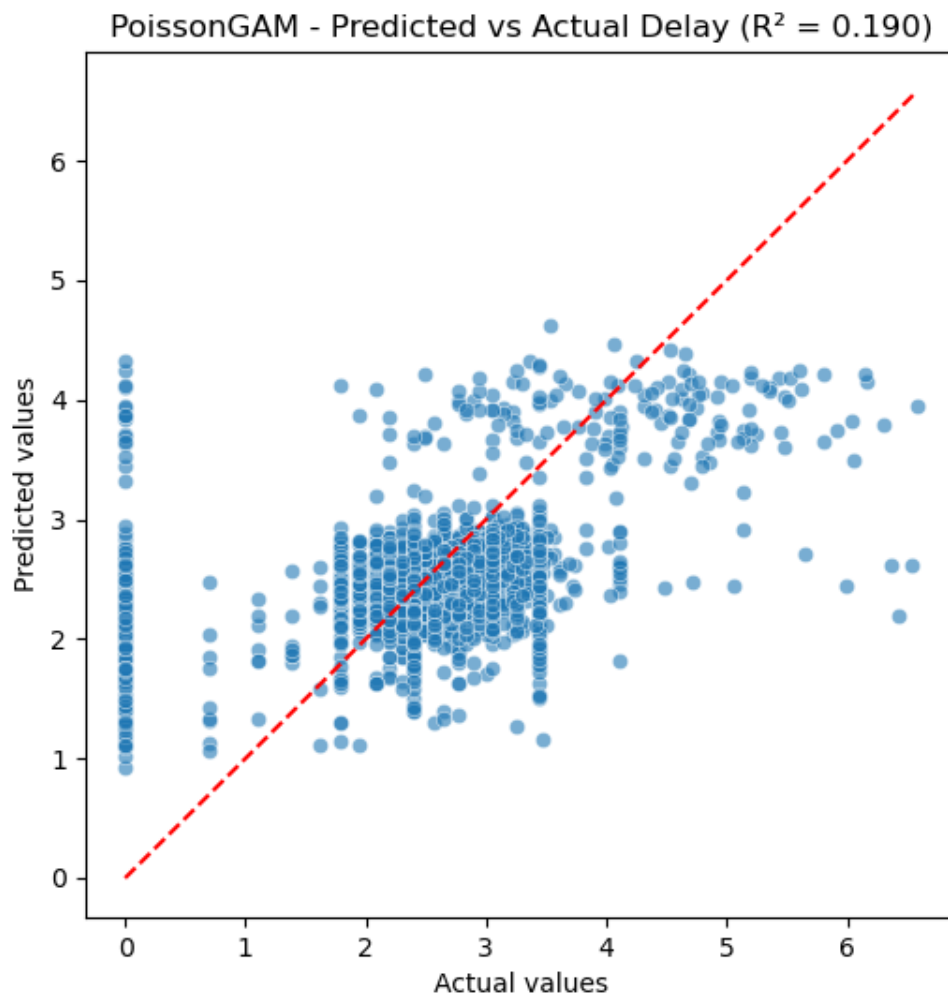
Results :

```

params = {
    "n_splines": 22,
    "lam": 1.4981548572944945,
    "max_iter": 495,
    "tol": 3.8603138507428227e-05
}

```

- **R2** : 0.1899
- **MAE** : 0.5833
- **RMSE** : 0.7444



Scenario 6 - GAMs, Summer

- **Entry data :** 4_preprocessed_dataset_pygam_summer.csv
- **Script :** train_pygam_summer.py
- **Train/Test/Validation split :** Train (80%)/Validation (10%)/Test (10%)

With the following data specifications :

```
generic_cols_transf = (
    f(0) + # ROUTE - nominal
    f(1) + # INCIDENT - nominal
    s(2) + # VISIBILITY - ordinal
    f(3) + # Clear - binary
    f(4) + # Fog - binary
    f(5) + # Rain - binary
    # f(6) + No Snow (all the data is False)
    f(6) + # Thunderstorms - binary
    s(7, basis="cp", edge_knots=[0, 24]) + # LOCAL_TIME_HOUR - cyclical
    s(8, basis="cp", edge_knots=[0, 60]) + # LOCAL_TIME_MINUTE - cyclical
    s(9, basis="cp", edge_knots=[0, 7]) + # WEEK_DAY - cyclical
    s(10, basis="cp", edge_knots=[1, 12]) + # LOCAL_MONTH - cyclical
    s(11, basis="cp", edge_knots=[1, 31]) + # LOCAL_DAY - cyclical
    s(12, basis="cp", edge_knots=[0, 360]) + # WIND_DIRECTION - cyclical
    f(13) + # PRECIP_AMOUNT_BINARY - binary
    s(14) + # TEMP - continuous
    s(15) + # DEW_POINT_TEMP - continuous
    s(16) + # HUMIDEX - continuous
    s(17) + # RELATIVE_HUMIDITY - continuous
    s(18) + # STATION_PRESSURE - continuous
    s(19) + # WIND_SPEED - continuous
)
```

LinearGAM Parameters :

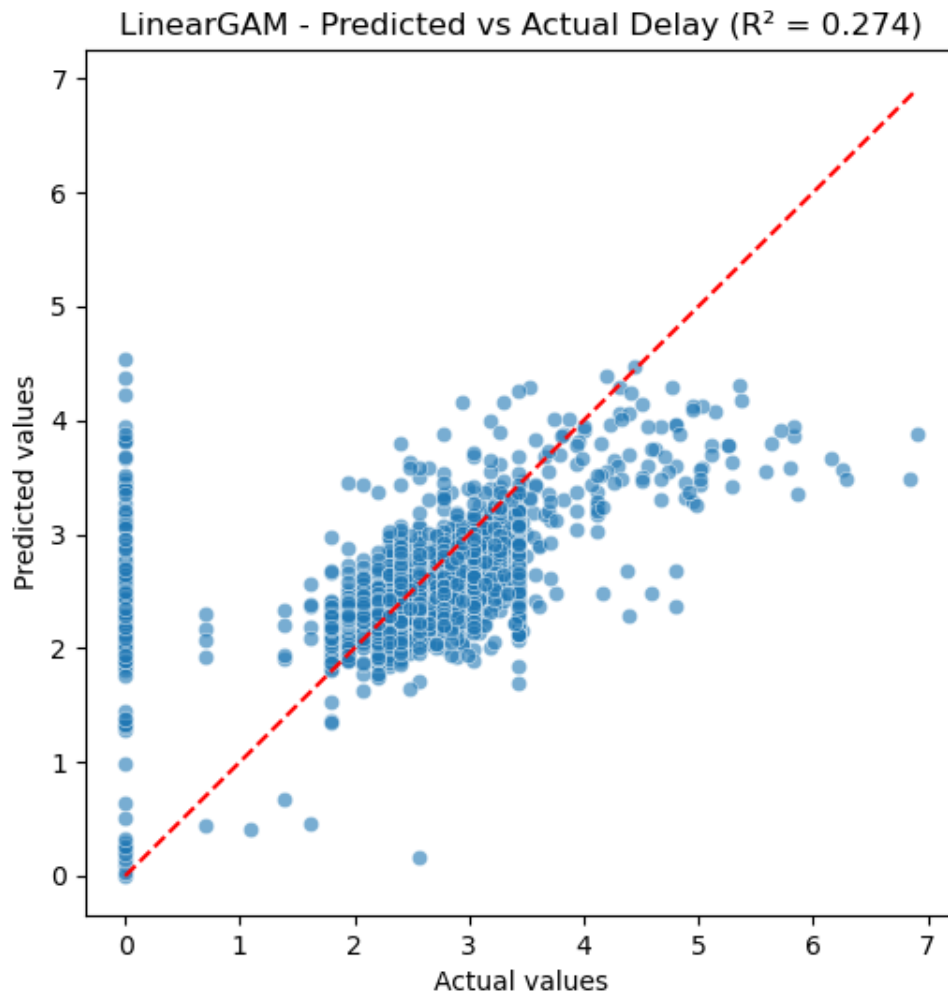
```
params = {
    "n_splines": n_splines,
    "lam": trial.suggest_float("lam", 1e-6, 1e3, log=True),
    "max_iter": trial.suggest_int("max_iter", 100, 1000),
    "tol": trial.suggest_float("tol", 1e-6, 1e-3, log=True),
}
```

Results :

```
params = {
    "n_splines": 7,
    "lam": 0.17608365605896287,
    "max_iter": 362,
    "tol": 3.204106745231952e-06
}
```

- **R2 :** 0.2742
- **MAE :** 0.4608

- RMSE : 0.6160



GammaGAM Parameters :

```
params = {
    "n_splines": trial.suggest_int("n_splines", 5, 30),
    "lam": trial.suggest_float("lam", 1e-6, 1e3, log=True),
    "max_iter": trial.suggest_int("max_iter", 100, 1000),
    "tol": trial.suggest_float("tol", 1e-6, 1e-3, log=True),
}
```

Results :

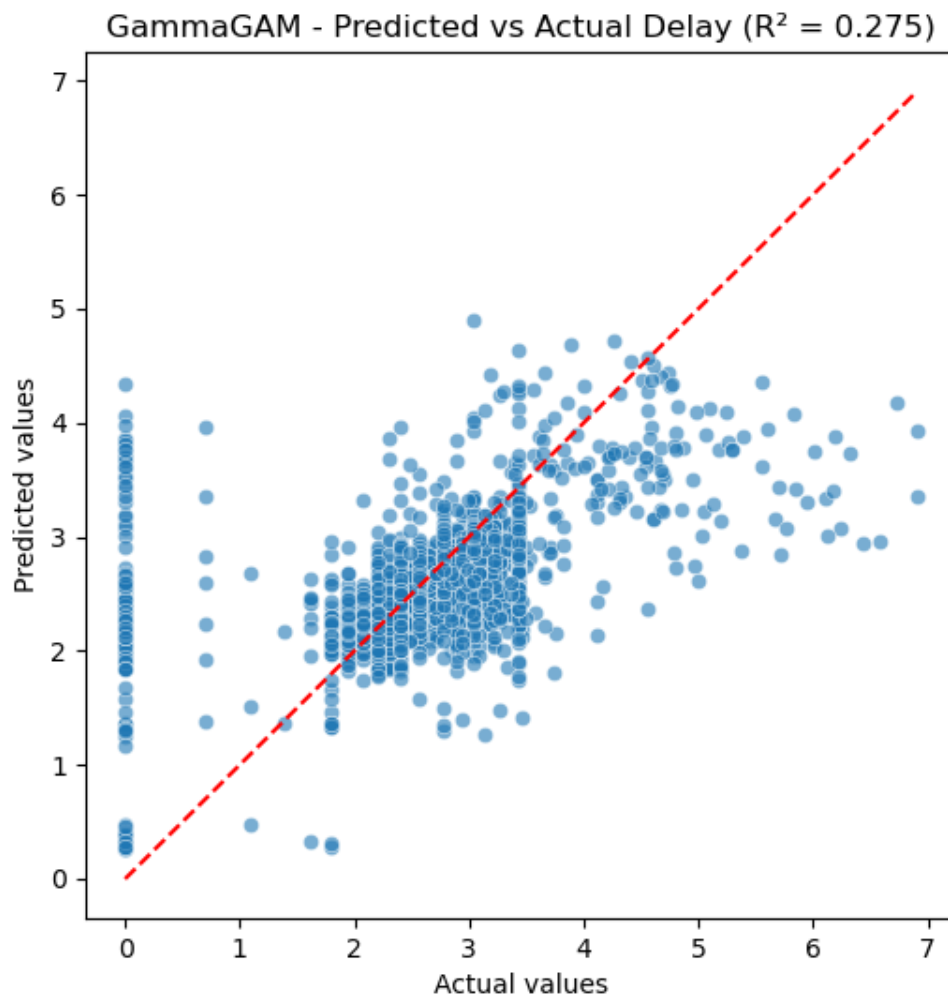
```
params = {
    "n_splines": 13,
    "lam": 4.951922066996407,
    "max_iter": 898,
```

```

    "tol": 2.6248532336832137e-05
}

```

- **R2** : 0.2746
- **MAE** : 0.4928
- **RMSE** : 0.6622



PoissonGAM We did not execute 100 trials due to long execution times. We executed only 25 tests with the following parameters.

Parameters :

```

params = {
    "n_splines": trial.suggest_int("n_splines", 5, 30),
    "lam": trial.suggest_float("lam", 1e-6, 1e3, log=True),
    "max_iter": trial.suggest_int("max_iter", 100, 1000),
    "tol": trial.suggest_float("tol", 1e-6, 1e-3, log=True),
}

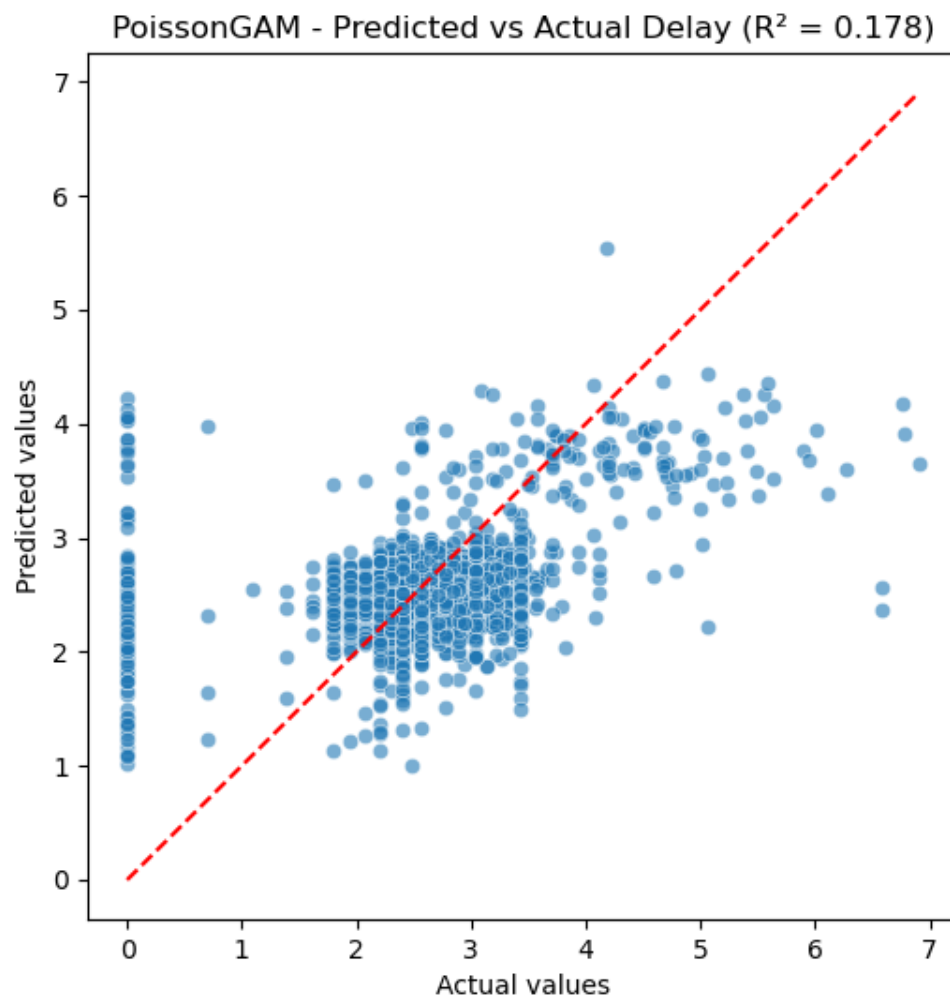
```

```
}
```

Results :

```
params = {  
  "n_splines": 21,  
  "lam": 0.27892287949265293,  
  "max_iter": 765,  
  "tol": 9.912575933470949e-06  
}
```

- **R2** : 0.1784
- **MAE** : 0.5582
- **RMSE** : 0.7279



1.3 Results summary

To end our testing and evaluations of the different models under different scenarios, here is a table summarizing our results.

R ² ; MAE ; RMSE	All seasons	Winter	Summer
Decision Tree	<i>0.2289 ; 0.5594 ; 0.7146</i>	0.1957 ; 0.5746 ; 0.7482	0.1615 ; 0.5799 ; 0.7578
Linear Regression (classical)	<i>0.2921 ; 0.4898 ; 0.6560</i>	0.2853 ; 0.5017 ; 0.6648	0.2826 ; <i>0.4824 ; 0.6484</i>
Linear Regression (elasticnet)	<i>0.2948 ; 0.4902 ; 0.6535</i>	0.2877 ; 0.5043 ; 0.6626	0.2865 ; <i>0.4874 ; 0.6449</i>
Random Forest	<i>0.0140 ; 0.6220 ; 0.9137</i>	-0.0009 ; 0.6217 ; 0.9310	-0.0002 ; <i>0.6112 ; 0.9040</i>
Gradient Boosting	<i>0.3002 ; 0.4465 ; 0.6485</i>	0.2625 ; 0.4650 ; 0.6860	0.2969 ; 0.4882 ; <i>0.6354</i>
Extreme Gradient Boosting	0.3296 ; 0.4746 ; 0.6212	0.3182 ; 0.4916 ; 0.6342	0.3245 ; <i>0.4712</i> ; 0.6106
Support Vector Regression	<i>0.2601 ; 0.4451 ; 0.6857</i>	0.2570 ; 0.4472 ; 0.6911	0.2053 ; 0.4885 ; 0.7183
LinearGAM	0.2958 ; 0.4896 ; 0.6526	<i>0.3142</i> ; 0.4947 ; 0.6515	0.2742 ; 0.4608 ; <i>0.6160</i>
GammaGAM	<i>0.2838 ; 0.4975 ; 0.6637</i>	0.2781 ; 0.4929 ; <i>0.6372</i>	0.2746 ; <i>0.4928</i> ; 0.6622
PoissonGAM	<i>0.2179 ; 0.5648 ; 0.7248</i>	0.1899 ; 0.5833 ; 0.7444	0.1784 ; <i>0.5582</i> ; 0.7279

To facilitate the reading and comparaisn on the table. The best values per column will be in **bold** and the best values per line will be in *italic*.

It appears that the model that performs best is **Extreme Gradient Boosting** on winter and summer, with SVR close behind it. Overall, extreme gradient boosting has it's most balanced results while being trained on all seasons. So that's what we will pick for our choice of model.

However, no model has good enough performances, we have to keep in mind that the scores are calculated with a log transformation, making the real scores much worst. Furthermore, all the models have approximatley the same performance (not much variation in results except for Random Forest), this may be a sign that something is wrong in our way to pre-process the data or the data itself.

1.4 Conclusion

From the bivariate analysis we had done in the EDA, we could have predicted this result. Since there are very few correlations in between our features and our target variable, we weren't able to produce a model capable of accurately predicting our target.

Perhaps we should have processed our target variable differently, maybe we didn't tranform our features correctly. Currently, except for the fact that we have very few correlations with our target,

we don't have another explanation as to why we weren't able to produce a good model for predicting the delay Toronto's bus routes from the weather conditions.

However, we did try to do our best to produce a model by doing data collection, extensive analysis on the found data and multiple tests of different models across multiple scenarios. Our conclusion, is not that it is impossible to predict the delays on Toronto's bus routes but rather that we didn't succeed even though we gave it our best effort.

We ended up by training and exporting the Extreme Gradient Boosting model for all seasons with it's hyperoptimized parameters and exported a fitted transformer to be able to predict the bus routes in the web application (both were exported as `preprocessor.pkl` and `xgb_model.pkl`).

1.5 Final notes

We did some tests that we didn't documented in this report because we didn't keep the results for different reasons. This section aims to give you an additionnal view on what we did during this project and what we tested.

- Tried to optimize models by maximizing/minimizing R^2 /MAE. The results were very similar to what we acheived with RMSE, in general slightly worst.
- Tried to scale our variable using MinMax scaling. Gave us approximately the same results as standard scaling.
- Tried to scale our variable using the Yeo-Johnson transformation. Gave us approximately the same results as standard scaling and MinMax scaling.
- Removing all the data that had an extremely high delay. We actually started trying this solution by removing all the data in which the delay exceeded the 95th percentile. It seemed to give us better results, however these results remained unsatisfying for an actual prediction. Since we found these last minutes results interesting, we created the files `preprocessor_opt.pkl` and `xgb_model_opt` to keep the option to use it in the app.

Here are some other ideas we had but unfortunately didn't test enough because of the lack of time.

- Testing general predictions, independant of route types and/or incidents
- Testing the prediction of the incident rather than the delay
- Testing more extensively the removal of outliers